CODERHOUSE · AI AUTOMATION

Entrega Final

Ecosistema autónomo de automatización con IA para generar contenido de Instagram sobre trámites automotores en Argentina

Dato	Detalle
Establecimiento	Coderhouse
Curso	AI Automation
Alumno	Ricardo Josa
Fecha	16 de octubre de 2026
Stack	n8n (orquestador) · OpenAI (gpt-5-mini + embeddings) · Airtable (base de datos) · Slack y Gmail (canales) · Notion (base de conocimiento RAG)

Accesos rápidos

Recurso	Enlace
Video demo (3 min)	https://youtu.be/dMU-k2jlhec
Dashboard de KPIs (Shared View pública)	https://airtable.com/appJBMsP4mvkNRtX8/shrcJqNW9fhIaMOKn/tblV3hHeQrxBDNDMO
Base de datos en modo lectura · Content	https://airtable.com/appJBMsP4mvkNRtX8/shryemI2kjEcNvxUC/tblWZq3FN7hiCDuA7
Base de datos en modo lectura · Log de Errores	https://airtable.com/appJBMsP4mvkNRtX8/shrHJnO3isSTlKYFc/tbls7kY8zIuQFuMat
Base de datos en modo lectura · Solicitantes (sin e-mail)	https://airtable.com/appJBMsP4mvkNRtX8/shrYu4IXhxmQBEDdP/tblzFzbB0j3b3Yv8G
Blueprint JSON · flujo principal (Drive)	https://drive.google.com/file/d/12SIWAtjaGD0JcJbtNbn9fmbJ5SbNKorn/view?usp=sharing
Blueprint JSON · Error Workflow (Drive)	https://drive.google.com/file/d/1kjR486NlzxmM_Rlqi5NV_Xrv6ebbM7Jy/view?usp=sharing

Contenido

1. Resumen ejecutivo
2. Entregable 1 · Arquitectura del ecosistema y demo
3. Entregable 2 · Manual operativo de datos (esquema relacional, esquemas JSON y operación)
4. Entregable 3 · Matriz de costos y selección de modelos
5. Entregable 4 · Seguridad y resiliencia
6. Entregable 5 · Dashboard de control (KPIs)
7. Stress test y evidencias de ejecución
8. Matriz de cumplimiento de la consigna
9. Evolución hacia producción (fuera del alcance de esta entrega)

Anexo · Blueprints JSON (links a Drive)

1. Resumen ejecutivo

Una gestoría automotor necesita publicar contenido educativo en Instagram (transferencias, verificación policial, formularios, costos, mitos y errores frecuentes) sin dedicar horas de redacción ni arriesgarse a publicar datos incorrectos. El ecosistema resuelve ese problema de punta a punta: un colaborador pide un post desde Slack, un agente de IA valida el tema contra una base de conocimiento propia (RAG sobre Notion), redacta el borrador, lo registra en Airtable, lo envía a un revisor humano por Gmail y, solo con aprobación explícita, simula la publicación y avisa al solicitante en el mismo hilo de Slack.

Qué cubre esta entrega respecto de la consigna:

Requisito técnico	Cómo se resolvió	Dónde verlo
Trigger inteligente	Slack Trigger por canal + filtro de 3 condiciones (persona, canal, sin subtype) + validación de solicitante y de texto antes de llamar a la IA.	§2.2 · Fig. 7
Error handling con registro en DB	Error output en el AI Agent, validaciones con Set de error y un Error Workflow global. Todo termina en la tabla Log de Errores.	§5.2 · §7
HITL antes de la acción crítica	Gmail Send & Wait con Aprobar/Rechazar y doble verificación en Airtable antes de publicar.	§5.3 · Fig. 12
Mapeo de Thread ID de Slack	`thread_ts || ts` en Config, guardado en `Slack_Thread_TS`; todas las respuestas van al hilo.	§3.3 · Fig. 11
Sin datos hardcodeados	Nodo Config (flujo principal) y Config Errores (Error Workflow) centralizan canales, e-mails, IDs y URL.	Fig. 6
Filtro anti-loop	Descarta mensajes del bot (`bot_id`), de otros canales y eventos de sistema (`subtype`: borrados, ediciones).	Fig. 7 · Fig. 30
Comparaciones de tipo correctas	Todos los If con validación estricta: boolean `is true`, string `equals`/`not empty`, `exists`.	§5.4
Prompt dinámico	El Prompt (User Message) combina variables del sistema: nombre del solicitante (Airtable), fecha actual y texto del mensaje (Slack). La memoria por usuario conserva el contexto.	Fig. 8
Stress test (5+ ejecuciones)	10 escenarios: 4 caminos felices/rechazo y 6 caminos no felices, más una prueba de trazabilidad del dato.	§7

> **Resultado del stress test:** 4 piezas generadas (3 publicadas, 1 rechazada por el revisor), 4 incidentes registrados (2 Datos Faltantes, 1 Fallo API, 1 Fallo AI) y 0 publicaciones sin aprobación humana. Los incidentes fueron provocados a propósito para validar cada rama de error.

2. Entregable 1 · Arquitectura del ecosistema y demo

2.1 Diagrama de arquitectura

La arquitectura se presenta en dos vistas: la primera muestra qué sistemas intervienen y qué datos intercambian; la segunda, la lógica de decisión de un pedido con sus rutas de error.

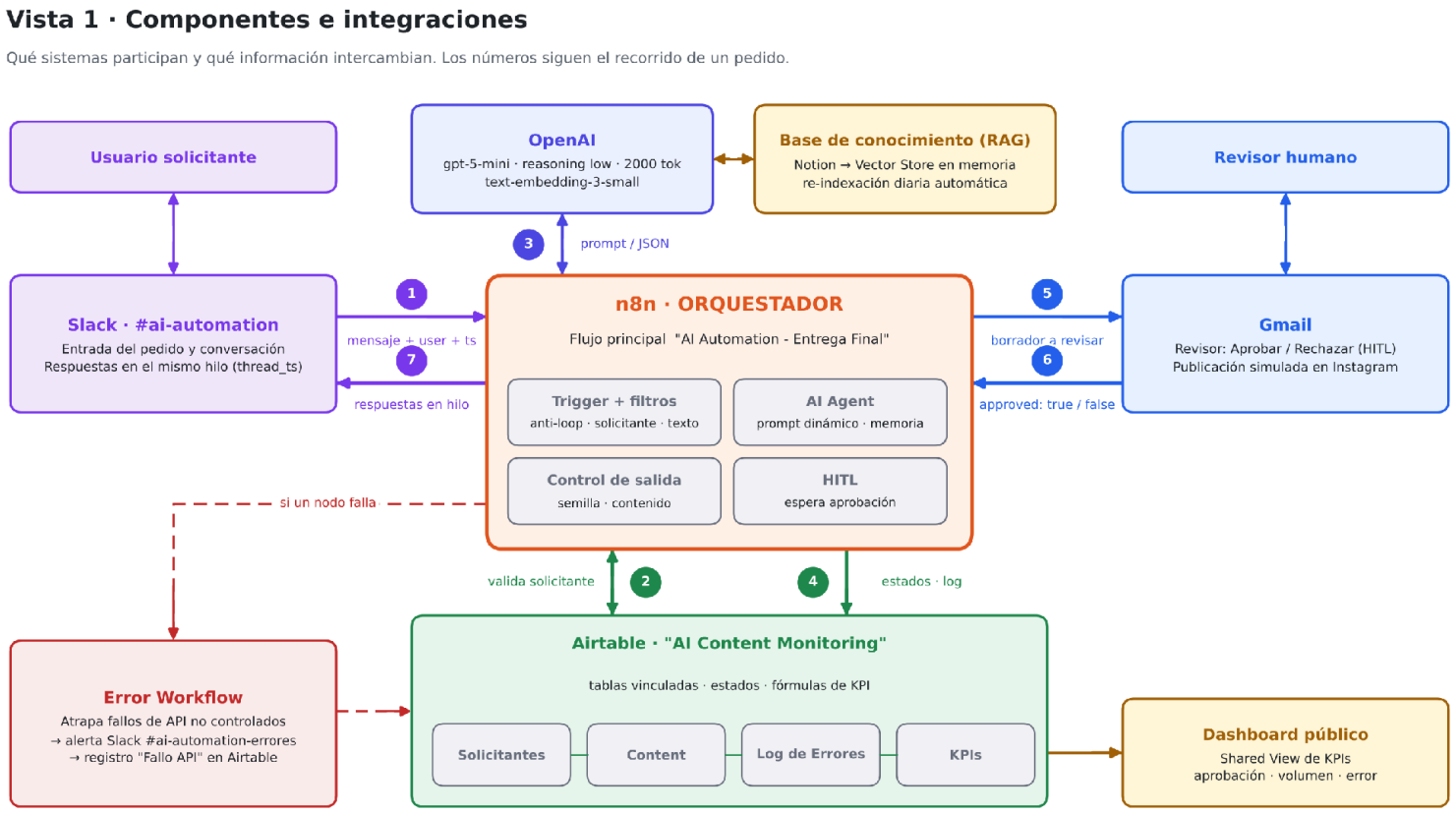


Figura 1. *Vista 1 · Componentes e integraciones: Slack (entrada y salida), n8n (orquestador), OpenAI + RAG, Gmail (HITL y publicación), Airtable (memoria y registro) y Error Workflow.*

Vista 2 · Flujo lógico de un pedido

Camino principal en línea recta (naranja). Cada decisión tiene su salida "no" hacia una ruta controlada. Todo error queda registrado en Log de Errores.

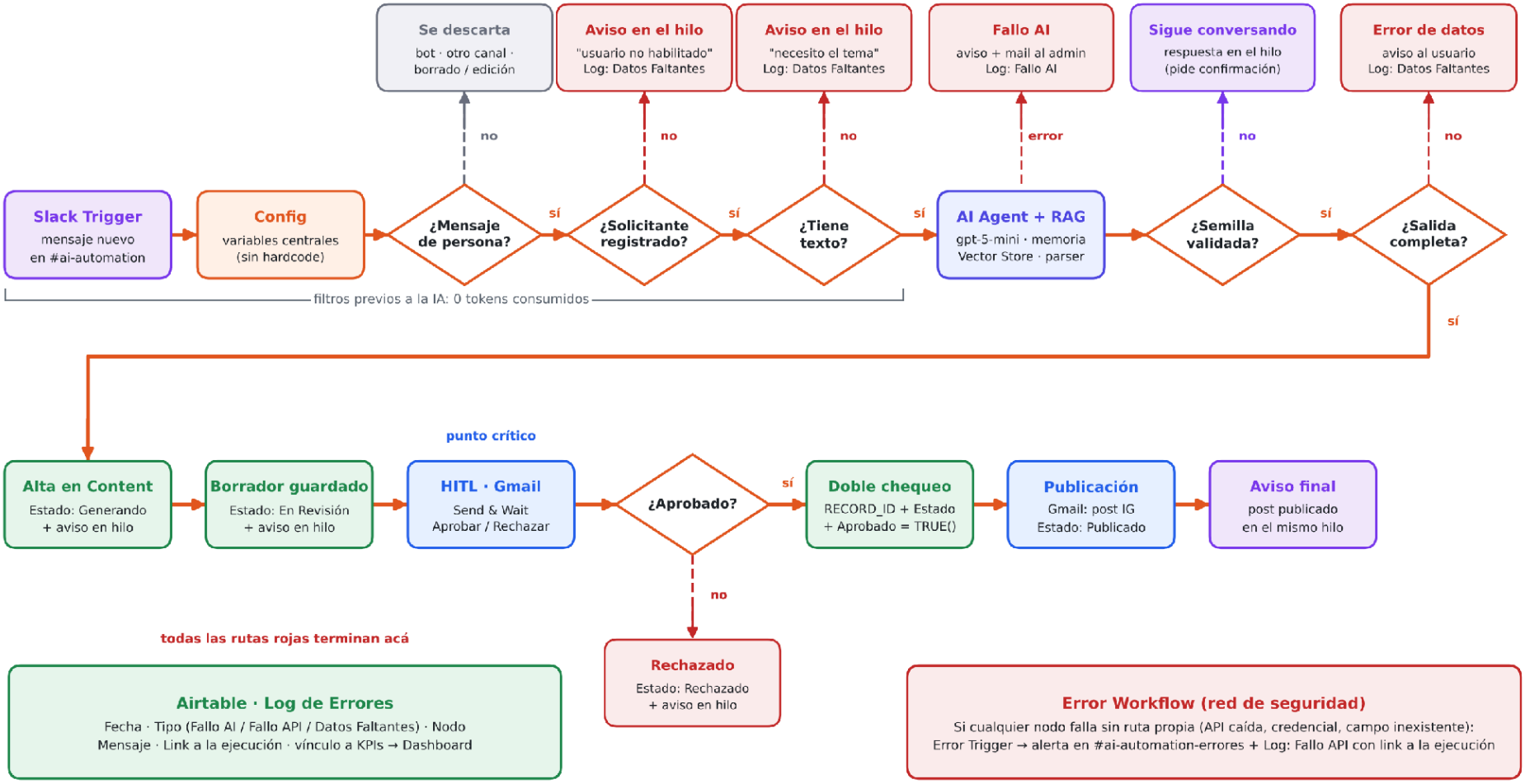


Figura 2. *Vista 2 · Flujo lógico: triggers y filtros, router de decisiones, nodo de IA, HITL, acción crítica y destino de cada ruta de error.*

2.2 Recorrido de una solicitud

Etapa	Nodos principales	Qué hace y por qué
0 · Ingesta RAG	Schedule Trigger (diario, 00:00) + Manual Trigger → Notion (Get many) → Code → In-Memory Vector Store + Embeddings	Carga la base de conocimiento de Notion (temas atómicos) en un vector store con `memoryKey = notion_rag` y *Clear Store* activo. Se recarga sola todas las noches (toma los cambios de Notion y repone el índice si n8n se reinició); el Manual Trigger queda para forzar una recarga.
1 · Trigger y validaciones	Slack Trigger → Config → ¿Mensaje de Persona? → Busco Solicitante → ¿Solicitante Registrado? → ¿Mensaje con Texto?	Filtra todo lo que no es un pedido válido **antes** de la IA: mensajes del bot, otros canales, eventos de sistema, usuarios no habilitados y mensajes sin texto. Nada de esto consume tokens.
2 · Agente AI + RAG	AI Agent + OpenAI Chat Model (gpt-5-mini) + Simple Memory + Simple Vector Store (tool) + Structured Output Parser	Conversa en dos etapas: valida la semilla contra la base (pide confirmación explícita) y luego redacta el post usando solo información recuperada. Devuelve JSON estructurado.
3 · Validación de salida	¿Semilla Validada? → ¿Salida AI Completa?	Si todavía está conversando, responde en el hilo. Si dice "validado" pero falta semilla o contenido, lo trata como error de datos.
4 · Registro y aviso	Inserto Registro (Generando) → Wait 10 s → Cambio de Estado (En Revisión)	Crea la pieza en Content vinculada al solicitante y a KPIs; guarda el borrador. El Wait separa los dos avisos al usuario.
5 · HITL	Envío Mail de Revisión (Send & Wait) → ¿Revisión Aprobada?	El revisor aprueba o rechaza desde el mail. El flujo queda en pausa hasta 24 h.
6 · Acción crítica	Actualizo Aprobado → Busco Registro Aprobado → Simula Publicación (Gmail) → Actualizo Publicado → Aviso final	Antes de publicar, vuelve a leer el registro y exige `RECORD_ID`, `Estado = Aprobado` y `Aprobado = TRUE()`. Si se rechaza, actualiza el estado y avisa.
7 · Errores in-flow	Error: Fallo AI / Usuario No Registrado / Mensaje Vacío / Salida AI Incompleta → Registro de Log de Errores	Cada rama arma `tipo · nodo · mensaje` en un Set y todas convergen en un único nodo de registro.
8 · Error Workflow	Error Trigger → Config Errores → Slack #ai-automation-errores + Registro de Fallo API	Atrapa cualquier fallo no controlado (APIs caídas, credenciales, campos inexistentes) y lo notifica con el link a la ejecución.

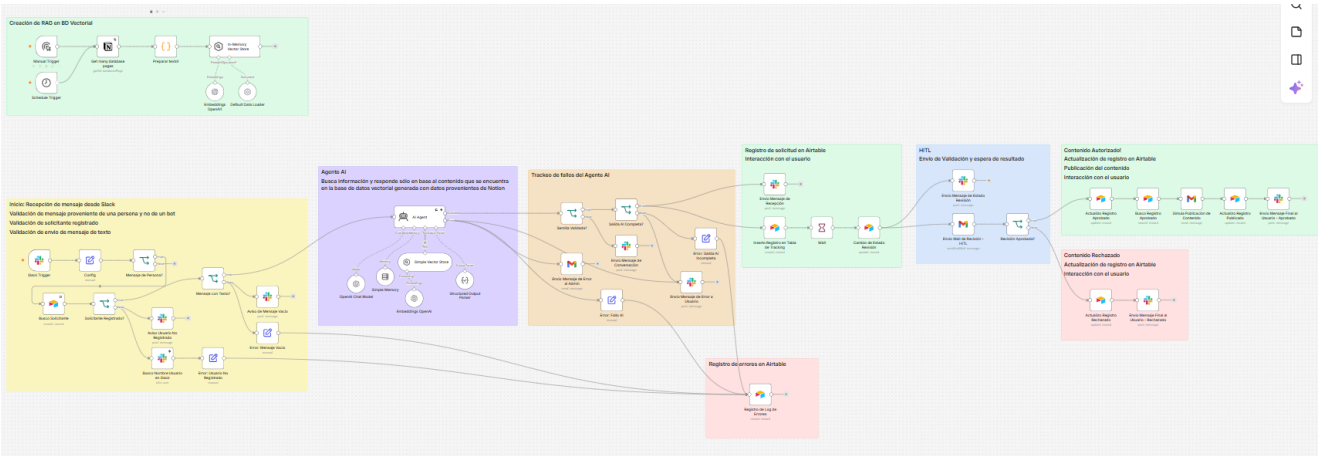


Figura 3. *Lienzo del flujo principal en n8n, organizado por etapas con notas de color.*

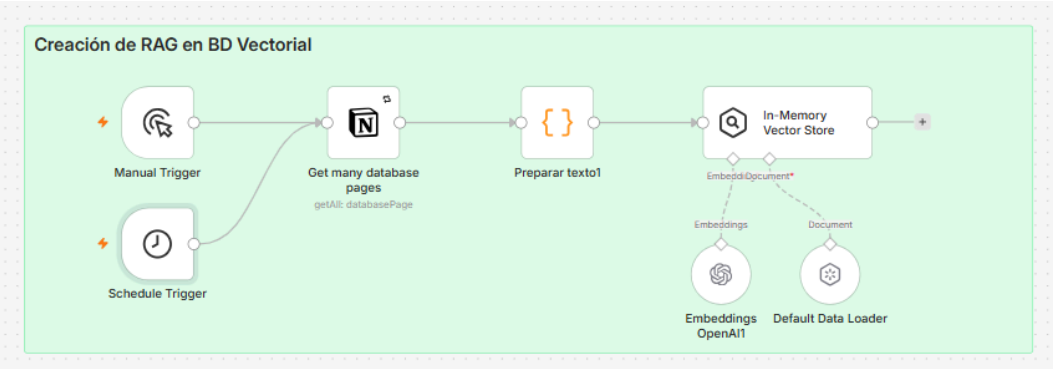


Figura 4. *Ingesta RAG: Schedule Trigger diario y Manual Trigger alimentan la carga de Notion al vector store.*

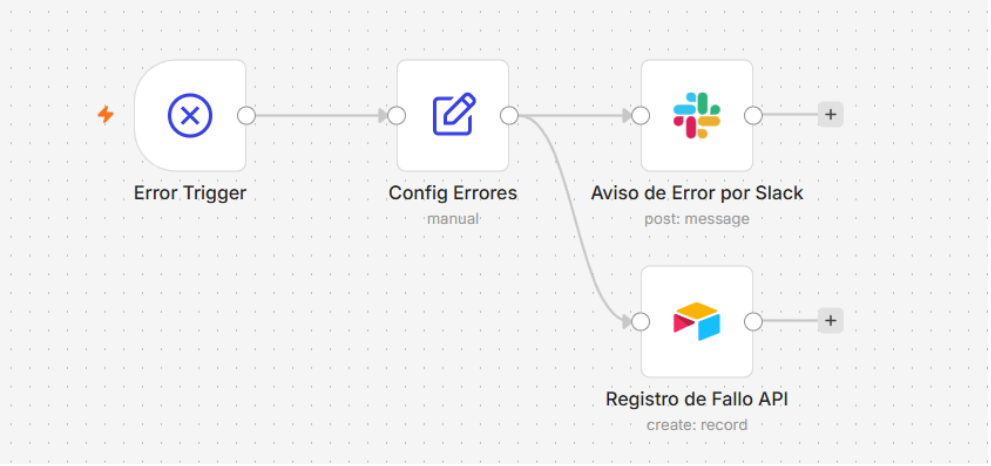


Figura 5. *Error Workflow "AI Automation - Entrega Final - Manejo de Errores".*

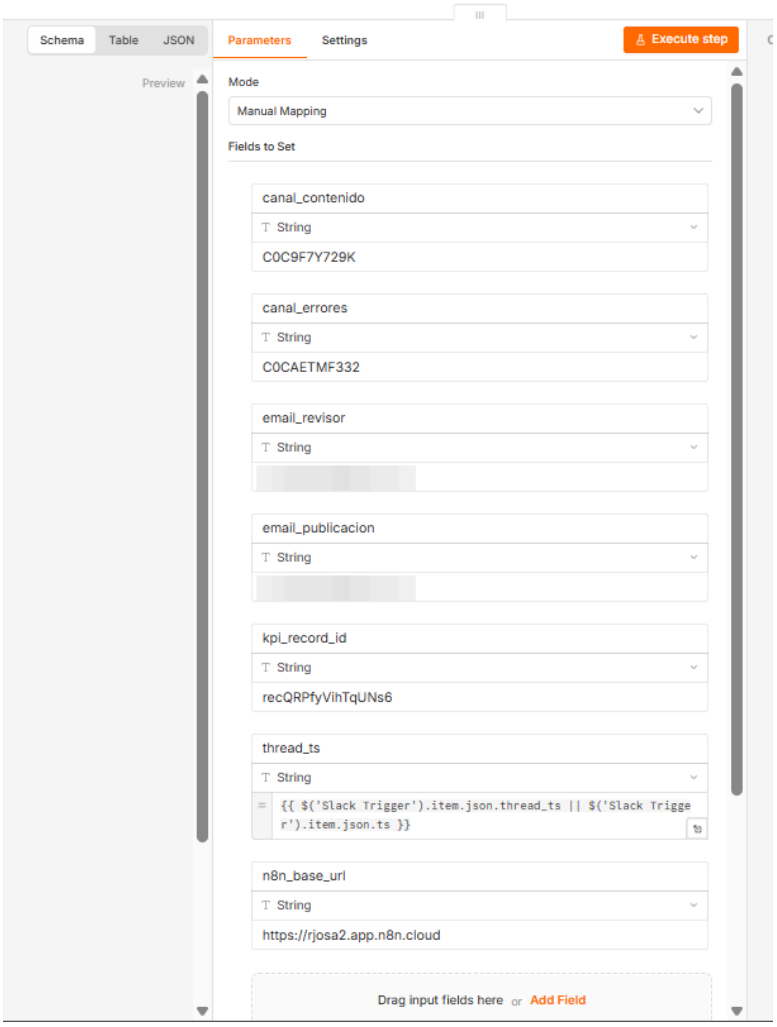


Figura 6. *Nodo Config: variables centralizadas (canales, e-mails, ID del registro de KPIs, thread_ts y URL base de n8n).*

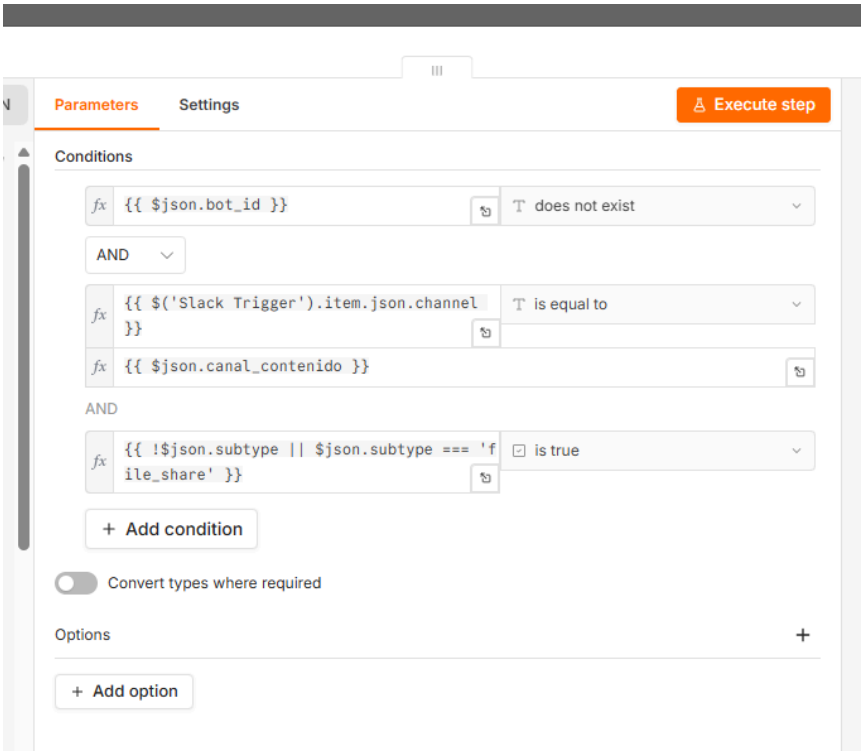


Figura 7. *Trigger inteligente: filtro anti-loop con tres condiciones y validación estricta de tipos.*

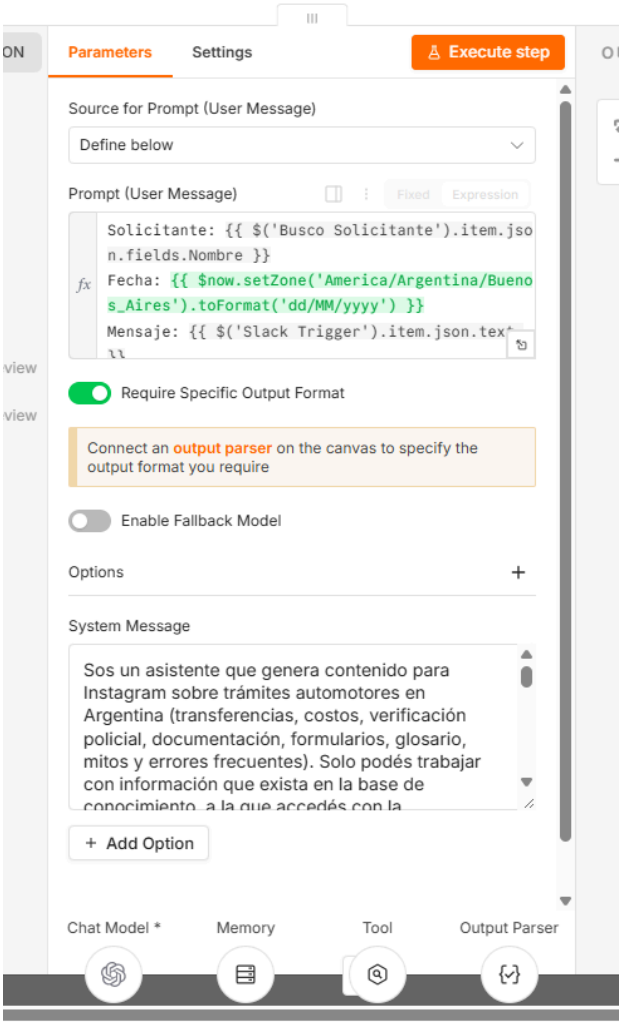


Figura 8. *AI Agent: prompt dinámico con variables del sistema (Solicitante, Fecha, Mensaje) y system message estático con las reglas de las dos etapas.*

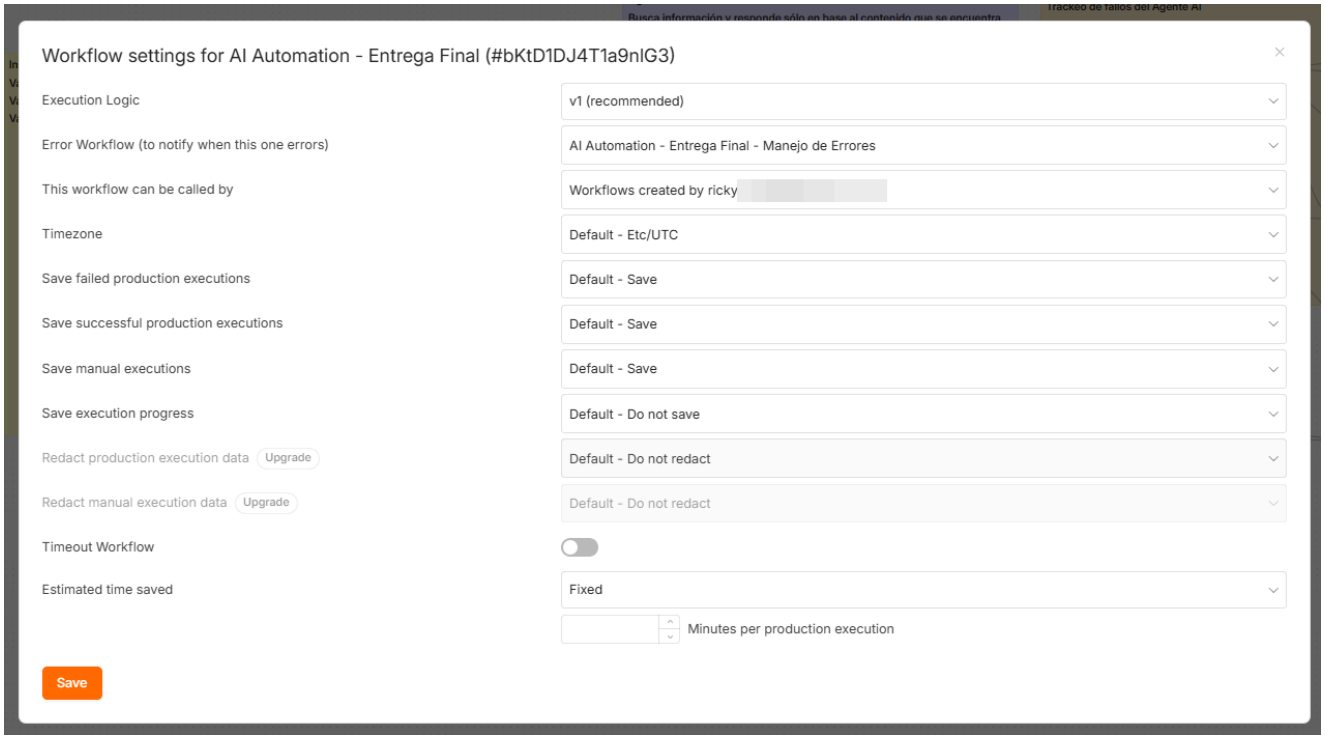


Figura 9. *Settings del flujo principal con el Error Workflow asignado.*

> **Alcance de la publicación:** la distribución final en Instagram está **simulada**. El nodo "Simula Publicación de Contenido" envía el post aprobado por Gmail a la casilla de publicación (`email_publicacion`), y el estado "Publicado" en Airtable significa "entregado para publicar". Reemplazarlo por la Graph API de Instagram queda como evolución hacia producción (§9), fuera del alcance de esta entrega.

2.3 Video demo

Link del video:

https://youtu.be/dMU-k2jlhec

En la grabación no se muestran API keys ni credenciales: el flujo usa credenciales OAuth2 guardadas en n8n y la credencial administrada del AI Gateway, que no exponen secretos en pantalla.

3. Entregable 2 · Manual operativo de datos

3.1 Modelo relacional

La base "AI Content Monitoring" de Airtable tiene cuatro tablas vinculadas con campos de tipo **Vincular a otro registro**. El ID autonumérico es para lectura humana; para la API se usa siempre el record id (`rec…`).

Relación	Cardinalidad	Para qué sirve
Solicitantes → Content	1 : N	Cada pieza queda asociada a quien la pidió (trazabilidad y control de acceso).
Content → Log de Errores	1 : N	Permite asociar incidentes a una pieza (campo reservado `Content_ID`).
Content → KPIs	N : 1	Todas las piezas se vinculan al registro "Global" para calcular indicadores.
Log de Errores → KPIs	N : 1	Todos los incidentes se vinculan al registro "Global".

Tabla Solicitantes

Campo	Tipo	Uso
ID	Número automático	Identificador legible.
Nombre	Texto (campo principal)	Nombre visible del solicitante.
e-mail	Correo electrónico	Dato de contacto. **Oculto en la vista pública** (minimización).
Fecha_de_Actualizacion	Fecha y hora (GMT-3)	Última modificación del registro.
Slack_User_ID	Texto	Clave de búsqueda contra `user` del evento de Slack.
Content	Vínculo → Content	Piezas pedidas por el solicitante.

Tabla Content (tracking de piezas)

Campo	Tipo	Uso
ID	Número automático (principal)	Número de pieza.
Tema	Texto	Semilla confirmada por el usuario (`output.semilla`).
Solicitante	Vínculo → Solicitantes	Record id devuelto por Busco Solicitante.
Estado	Selección única	Generando · En Revisión · Aprobado · Rechazado · Publicado · Error.
Fecha_de_Creacion	Fecha y hora (GMT-3)	Alta de la pieza (`$now.setZone(...).toISO()`).
Borrador_Generado	Texto largo	Post generado por la IA (`output.contenido`).
Aprobado	Casilla de verificación	Se marca solo con aprobación HITL; se exige en el filtro final.
Fecha_de_Actualizacion	Fecha y hora (GMT-3)	Se actualiza en cada cambio de estado.
Slack_Thread_TS	Texto	ID del hilo de Slack para responder siempre en la misma conversación.
KPI	Vínculo → KPIs	Vínculo al registro "Global".
Log_de_Errores	Vínculo → Log de Errores	Incidentes asociados a la pieza.

Tabla Log de Errores

Campo	Tipo	Uso
ID	Número automático (principal)	Número de incidente.
Fecha	Fecha y hora (GMT-3)	Momento del incidente.
Tipo	Selección única	Fallo AI · Fallo API · Datos Faltantes.
Nodo	Texto	Nodo donde se originó.
Mensaje	Texto largo	Detalle técnico o descripción legible.
Link_Ejecucion	URL	`n8n_base_url/workflow/{id}/executions/{id}` o `execution.url` del Error Trigger.
Content_ID	Vínculo → Content	Reservado para asociar incidentes ocurridos después del alta de la pieza (ver §5.2).
KPIs	Vínculo → KPIs	Vínculo al registro "Global".

Tabla KPIs (registro único "Global")

Campo	Tipo	Cálculo
Nombre	Texto (principal)	"Global".
Total_Piezas	Recuento	Cantidad de registros vinculados de Content.
Publicadas	Compilación con condición	COUNTA de Content donde `Estado = Publicado`.
Rechazadas	Compilación con condición	COUNTA de Content donde `Estado = Rechazado`.
Total_Errores	Recuento	Cantidad de registros vinculados de Log de Errores.
Tasa_Aprobacion	Fórmula (%)	`{Publicadas} / {Total_Piezas}`
Volumen_Salida	Fórmula	`{Publicadas}`: piezas que llegaron a publicarse.
Tasa_Error	Fórmula (%)	`{Total_Errores} / ({Total_Piezas} + {Total_Errores})`: incidentes sobre el total de eventos procesados.

3.2 Ciclo de vida de una pieza (estados)

Estado	Lo asigna	Momento
Generando	Inserto Registro en Tabla de Tracking	La IA devolvió una salida completa y validada.
En Revisión	Cambio de Estado Revisión	Se guardó el borrador y se envió el mail HITL.
Aprobado	Actualizo Registro Aprobado	El revisor hizo clic en Aprobar (`Aprobado = true`).
Publicado	Actualizo Registro Publicado	Pasó el filtro final y se simuló la publicación.
Rechazado	Actualizo Registro Rechazado	El revisor hizo clic en Rechazar.

Las fechas se envían como `$now.setZone('America/Argentina/Buenos_Aires').toISO()` y los campos de fecha de Airtable usan zona fija Buenos Aires con hora, para evitar el corrimiento de un día que produce interpretar una fecha sin hora como medianoche UTC.

3.3 Esquemas JSON de transferencia

Estos son los objetos que viajan entre sistemas en cada etapa. Se muestran solo los campos que el flujo usa; los valores son reales de las ejecuciones de prueba (textos largos abreviados).

A · Evento entrante de Slack (Slack Trigger)

```
{
  "type": "message",
  "user": "U0C398HR015",              // clave para buscar al solicitante y para la memoria
  "text": "Verificación Policial",     // entra al prompt del AI Agent
  "ts": "1792153308.568579",          // ID del mensaje
  "thread_ts": "1792153308.568579",   // presente si el mensaje está dentro de un hilo
  "channel": "C0C9F7Y729K",           // debe coincidir con Config.canal_contenido
  "bot_id": null,                     // si existe → mensaje del bot → se descarta
  "subtype": null                     // message_deleted / message_changed → se descarta
}
```

Mapeo del hilo: `thread_ts = thread_ts || ts`. Si el mensaje abre una conversación, su `ts` pasa a ser el ID del hilo; si ya está dentro de uno, se reutiliza `thread_ts`. Ese valor se guarda en `Content.Slack_Thread_TS` y se usa en todas las respuestas.

B · Salida del nodo Config (variables de entorno del flujo)

```
{
  "canal_contenido": "C0C9F7Y729K",
  "canal_errores":   "C0CAETMF332",
  "email_revisor":   "<revisor>",
  "email_publicacion": "<publicación>",
  "kpi_record_id":   "recQRPfyVihTqUNs6",
  "thread_ts":       "{{ $('Slack Trigger').item.json.thread_ts || $('Slack
Trigger').item.json.ts }}",
  "n8n_base_url":    "https://rjosa2.app.n8n.cloud"
}
```

C · Salida estructurada del AI Agent (Structured Output Parser)

```
{
  "output": {
    "estado": "validado",                        // "conversando" | "validado"
    "respuesta": "El tema quedó confirmado...",   // mensaje breve para el hilo
    "semilla": "Verificación policial para venta de autos (requisitos, ...)",
    "contenido": "Verificación policial para la venta de autos: qué es y cómo hacerlo ✅ ..."
  }
}
```

Mientras `estado = conversando`, `semilla` y `contenido` llegan vacíos y solo se responde en el hilo. Con `estado = validado`, ambos deben venir con texto; si no, se registra un error de tipo Datos Faltantes.

D · Alta en Content (Airtable create)

```
{
  "fields": {
    "Tema": "Verificación policial para venta de autos (...)",
    "Solicitante": ["rec<id del solicitante>"],
    "Estado": "Generando",
    "Fecha_de_Creacion": "2026-10-16T09:22:05.412-03:00",
    "Fecha_de_Actualizacion": "2026-10-16T09:22:05.412-03:00",
    "Slack_Thread_TS": "1792153308.568579",
    "KPI": ["recQRPfyVihTqUNs6"]
  }
}
// respuesta: { "id": "rec…", "createdTime": "…", "fields": { … } }  → el id se reutiliza en los
updates
```

E · Paso a revisión (Airtable update)

```
{
  "id": "{{ $('Inserto Registro en Tabla de Tracking').item.json.id }}",
  "fields": {
    "Estado": "En Revisión",
    "Borrador_Generado": "{{ $('AI Agent').item.json.output.contenido }}",
    "Fecha_de_Actualizacion": "2026-10-16T09:22:15.880-03:00"
  }
}
```

F · Respuesta del HITL (Gmail Send & Wait)

```
{ "data": { "approved": true } }     // false si el revisor elige "Rechazar"
```

El If "Revisión Aprobada?" usa el operador **boolean · is true** con validación estricta: solo `true` habilita la publicación.

G · Filtro previo a la acción crítica (Airtable search)

```
AND(
  RECORD_ID() = '{{ $('Cambio de Estado Revisión').item.json.id }}',
  {Estado} = 'Aprobado',
  {Aprobado} = TRUE()
)
```

H · Registro en Log de Errores (in-flow)

```
// Cada rama arma este objeto en un Set...
{ "tipo": "Fallo AI", "nodo": "AI Agent",
  "mensaje": "The AI model returned an empty response to the Structured Output Parser" }

// ...y el nodo único "Registro de Log de Errores" lo persiste:
{
  "fields": {
    "Fecha": "2026-10-16T12:20:48.000-03:00",
    "Tipo": "Fallo AI",
    "Nodo": "AI Agent",
    "Mensaje": "The AI model returned an empty response to the Structured Output Parser",
    "Link_Ejecucion": "https://rjosa2.app.n8n.cloud/workflow/bKtD1DJ4T1a9nlG3/executions/806",
    "KPIs": ["recQRPfyVihTqUNs6"]
  }
}
```

I · Payload del Error Trigger (Error Workflow)

```
{
  "execution": {
    "id": "804",
    "url": "https://rjosa2.app.n8n.cloud/workflow/bKtD1DJ4T1a9nlG3/executions/804",
    "lastNodeExecuted": "Busco Solicitante",
    "error": { "message": "Your request is invalid or could not be processed by the service" }
  },
  "workflow": { "id": "bKtD1DJ4T1a9nlG3", "name": "AI Automation - Entrega Final" }
}
// → Slack #ai-automation-errores  +  Log de Errores { Tipo: "Fallo API", Nodo, Mensaje,
Link_Ejecucion }
```

3.4 Mapeo de la respuesta de la IA

El AI Agent devuelve el JSON del parser dentro de `output`. Los nodos siguientes leen siempre por referencia explícita al nodo, nunca por posición:

Variable	Expresión en n8n	Destino
Estado de la conversación	`$json.output.estado`	If "Semilla Validada?"
Respuesta al usuario	`$('AI Agent').item.json.output.respuesta`	Slack (hilo)
Tema confirmado	`$('AI Agent').item.json.output.semilla`	Content.Tema · avisos en Slack
Post generado	`$('AI Agent').item.json.output.contenido`	Content.Borrador_Generado → mail HITL
Error de la IA	`$json.error`	Mail al admin · Log de Errores

3.5 Manual operativo (cómo se usa y se mantiene)

Tarea	Quién	Cómo
Pedir un post	Solicitante	Escribir en #ai-automation un saludo o el tema. Responder "sí / confirmo" cuando el agente reformula la semilla. Todo el seguimiento llega en el mismo hilo.
Aprobar o rechazar	Revisor	Abrir el mail "Generación de Contenido IG: …", leer el borrador y elegir Aprobar o Rechazar. Sin respuesta en 24 h no se publica.
Habilitar un usuario	Administrador	Agregar un registro en Solicitantes con Nombre y Slack_User_ID (perfil de Slack → ··· → Copiar ID de miembro).
Actualizar la base de conocimiento	Administrador	Editar la base de Notion. El Schedule Trigger la re-indexa automáticamente cada noche; si el cambio es urgente, ejecutar el Manual Trigger del bloque "Creación de RAG en BD Vectorial".
Cambiar canales, e-mails o entorno	Administrador	Editar solo los nodos Config y Config Errores. Ningún otro nodo tiene valores fijos.

Tarea	Quién	Cómo
Monitorear la operación	Administrador	Revisar el Dashboard público y el canal #ai-automation-errores. Cada registro de Log de Errores tiene el link a la ejecución para diagnosticar.
Atender un incidente	Administrador	Abrir Link_Ejecucion, ver el nodo que falló, corregir la causa (credencial, campo, servicio caído) y pedir al usuario que reintente en el hilo.

4. Entregable 3 · Matriz de costos

Criterio general: **la IA se usa solo donde aporta comprensión de lenguaje o redacción**. Todo lo que es ruteo, validación, persistencia y notificación se resuelve con nodos determinísticos de n8n, que no consumen tokens.

> **Moneda:** todos los importes de esta sección están expresados en **dólares estadounidenses (USD)**, la moneda en la que OpenAI y Anthropic publican sus precios.

4.1 Modelo o herramienta por tarea

Tarea	Modelo / herramienta	Justificación	Costo (USD)
Filtrar eventos (bot, canal, subtype)	If de n8n	Regla determinística; no requiere IA.	$0
Validar solicitante	Airtable Search + If	Búsqueda exacta por `Slack_User_ID`.	$0 en tokens
Validar que haya texto	If de n8n	Evita invocar la IA con mensajes vacíos.	$0
Conversar y validar la semilla	gpt-5-mini · reasoning low	Necesita entender el pedido y usar la herramienta RAG; el razonamiento bajo alcanza y reduce tokens de salida.	≈ $0.002 por pieza
Recuperar información (RAG)	text-embedding-3-small + In-Memory Vector Store (topK 5)	Embedding más económico; la base es chica y en memoria, sin costo de base vectorial externa.	< $0.00001 por consulta
Redactar el post	gpt-5-mini · reasoning low · Max Tokens 2000	Calidad de redacción suficiente para un post corto basado en datos recuperados; el tope de tokens acota el peor caso.	≈ $0.004 por pieza
Estructurar la salida	Structured Output Parser	Formatea en JSON dentro de la misma llamada; no agrega llamadas.	$0
Validar la salida	If de n8n	Control de campos vacíos sin re-consultar a la IA.	$0
Aprobar o rechazar	Revisor humano por Gmail	La decisión crítica queda en una persona.	≈ 3–5 min de revisión
Registrar, notificar, alertar	Airtable, Slack, Gmail	Integraciones nativas.	$0 en tokens
Ingesta de la base RAG	text-embedding-3-small	Re-indexación diaria automática (Schedule Trigger).	< $0.001 por día

4.2 Estimación de consumo por pieza

Supuesto: una pieza típica requiere tres turnos con el agente (saludo, propuesta de tema, confirmación y generación), con dos consultas al vector store. Incluye system prompt, memoria de la conversación, fragmentos recuperados y tokens de razonamiento.

Turno	Input (tokens)	Output (tokens)
Saludo / presentación	≈ 800	≈ 270
Propuesta de tema (tool call + respuesta con RAG)	≈ 3.400	≈ 430
Confirmación + generación (tool call + post)	≈ 4.800	≈ 1.300
Total por pieza	**≈ 9.000**	**≈ 2.000**

Modelo (USD por 1M tokens in / out)	Costo por pieza (USD)	100 piezas/mes (USD)	Comentario
gpt-5-mini ($0.25 / $2.00) · elegido	**≈ $0.006**	**≈ $0.63**	Buen equilibrio entre calidad, uso de herramientas y costo.
gpt-5 ($1.25 / $10.00)	≈ $0.031	≈ $3.13	5 veces más caro sin mejora perceptible para posts cortos con RAG.

Modelo (USD por 1M tokens in / out)	Costo por pieza (USD)	100 piezas/mes (USD)	Comentario
gpt-5-nano ($0.05 / $0.40)	≈ $0.001	≈ $0.13	Más barato, pero con mayor riesgo de fallar el formato estructurado o el uso de la herramienta; además figura como deprecado.
Claude Haiku 5.5 ($0.10 / $0.50)	≈ $0.002	≈ $0.19	Alternativa más económica y apta para tareas simples. No se eligió porque el flujo ya está probado con OpenAI vía AI Gateway de n8n; es candidata a evaluar.
Claude Sonnet 5.5 ($2 / $10)	≈ $0.038	≈ $3.80	Conviene para lectura densa (normativa extensa, PDFs de registros). Sobredimensionado para un post corto.
gpt-5-mini vía **Batch API** (−50 %)	≈ $0.003	≈ $0.31	Solo para procesos masivos sin conversación (ej. generar un calendario mensual de 30 posts de una vez, resultados en hasta 24 h).

Matriz de decisión: qué modelo por tipo de tarea

Tipo de tarea	Modelo recomendado	Por qué
Conversación + redacción con RAG (este flujo)	**gpt-5-mini**, reasoning low, Max Tokens 2000	Maneja tool calling y salida estructurada de forma confiable a bajo costo.
Clasificación o extracción simple (ej. detectar categoría del pedido)	gpt-5-nano o Claude Haiku 5.5	Tarea corta y acotada; el modelo chico alcanza.
Lectura de documentos largos y densos	Claude Sonnet 5.5	Mejor comprensión de textos extensos; se justifica solo si cambia la fuente de datos.
Generación masiva programada	Batch API (OpenAI o Anthropic)	50 % de descuento a cambio de no ser en tiempo real.
Embeddings / búsqueda semántica	text-embedding-3-small	El más económico; suficiente para una base chica.

Precios de referencia de OpenAI consultados el 16/10/2026: gpt-5-mini $0.25 / $2.00, gpt-5 $1.25 / $10.00, gpt-5-nano $0.05 / $0.40 por millón de tokens de entrada / salida; text-embedding-3-small $0.02 por millón; Batch API de OpenAI 50 % de descuento. Precios de Anthropic consultados la misma fecha: Claude Haiku 5.5 $0.10 / $0.50 y Claude Sonnet 5.5 $2 / $10. No se incluyen los costos fijos de las plataformas (n8n Cloud, Airtable, Slack, Google Workspace).

> **Ahorro estimado frente a usar el modelo grande (gpt-5): ≈ 80% en costo de IA.** El tope Max Tokens 2000 limita el peor caso de cada llamada a ≈ $0.004 de salida.

4.3 Ahorros adicionales por diseño

- **Validaciones antes de la IA:** mensajes de bots, ediciones, borrados, usuarios no habilitados y mensajes vacíos se descartan sin llamar al modelo. En el stress test, 2 de los 4 incidentes y todos los eventos del propio bot se resolvieron con costo de IA $0.
- **Prompt caching:** el system message es 100 % estático y las variables (solicitante, fecha, mensaje) van al final, en el mensaje del usuario. OpenAI aplica el cache automáticamente sobre el prefijo idéntico de cada llamada (desde 1.024 tokens), y con gpt-5-mini el input cacheado cuesta $0.025 por millón en lugar de $0.25 (−90 %). Como la memoria agrega los turnos al final, en los turnos 2 y 3 también se reutiliza el historial previo.
- **Memoria por usuario y prompt acotado:** el agente usa la herramienta RAG como máximo una vez por turno, por regla del system prompt, y no repite consultas innecesarias.
- **Vector store en memoria:** no hay costo mensual de una base vectorial externa. La re-indexación diaria cuesta menos de un centavo, ≈ $0.03 por mes.
- **Tiempo humano:** redactar un post investigado lleva ≈ 30–40 min; revisar el borrador lleva ≈ 3–5 min. Con 100 piezas por mes, eso libera ≈ 45–55 horas. Es una estimación.

5. Entregable 4 · Seguridad y resiliencia

5.1 Minimización de datos

Medida	Detalle
Solo los campos necesarios del evento	Del evento de Slack se usan `user`, `text`, `ts`/`thread_ts`, `channel`, `bot_id` y `subtype`. No se guardan adjuntos ni metadatos del workspace.
Identificación mínima del solicitante	Content guarda solo el vínculo al solicitante y el ID del hilo. El nombre real de Slack se consulta únicamente cuando hace falta enriquecer el log de un usuario no registrado (nodo con *Continue on error*: si falla, queda el ID).
Base pública sin datos personales	Se revocó el link de la base completa porque exponía e-mails. El acceso de lectura es por Shared View por tabla y la vista de Solicitantes oculta el e-mail. La tabla heredada "Content Items" no se comparte.
Credenciales fuera del flujo	Slack, Gmail, Airtable y Notion usan credenciales OAuth2 gestionadas por n8n; OpenAI usa la credencial administrada del AI Gateway. El JSON exportado contiene referencias, no secretos.
Acceso restringido	Solo usuarios registrados en Solicitantes pueden pedir contenido. El trigger escucha un único canal.
Contexto aislado por usuario	La memoria del agente usa como clave el ID de usuario de Slack: no se mezclan conversaciones entre personas.
Datos de la IA acotados a la base	El agente solo responde con información recuperada del RAG ("no inventes datos, montos ni plazos"). Ante un tema fuera de la base, lo dice y sugiere temas válidos.

5.2 Manejo de errores (dos niveles)

Equivalencia con las directivas de Make que pide la consigna:

Directiva	Equivalente en n8n	Dónde se usa
Resume (seguir con un valor alternativo)	*On Error → Continue (using error output)* y *Continue (regular output)*	AI Agent (sale por la rama de error y responde al usuario) · Busco Nombre Usuario en Slack (si falla, sigue con el ID).
Break (detener y registrar)	*On Error → Stop Workflow* + **Error Workflow**	Resto de los nodos de API (Airtable, Slack, Gmail): el fallo detiene la ejecución y el Error Workflow registra el incidente y alerta.
Reintento automático	*Retry on fail*	AI Agent, Embeddings OpenAI y Notion.

Incidente	Detección	Respuesta al usuario / equipo	Registro
Usuario no registrado	Busco Solicitante (Always Output Data) + If `id exists`	Aviso en el hilo: "tu usuario no está habilitado…"	Log · Datos Faltantes (con nombre de Slack)
Mensaje sin texto	If `text.trim()` not empty	Aviso en el hilo pidiendo el tema	Log · Datos Faltantes
Salida AI incompleta	If semilla y contenido not empty	Mensaje de error en el hilo	Log · Datos Faltantes
Fallo de la IA (API de OpenAI, timeout, JSON inválido)	AI Agent con **Retry on fail** + **error output**	Mensaje de error en el hilo + mail técnico al admin	Log · Fallo AI
Fallo de API no controlado (Airtable, Slack, Gmail…)	Error Workflow asignado en Settings (Error Trigger)	Alerta en #ai-automation-errores con nodo, error, fecha y link	Log · Fallo API (con URL de la ejecución)
Eventos que generan loops	If anti-loop (bot_id, canal, subtype)	Se descartan en silencio	No aplica
Revisor que no responde	Send & Wait con límite de 24 h	Sin aprobación explícita no se publica (fail-safe)	Queda en estado En Revisión

> Se usan dos mecanismos a propósito. Los errores esperables de la IA se controlan **dentro** del flujo (error output) para dar una respuesta amable y seguir. Los fallos de infraestructura se dejan subir al **Error**

Workflow, que centraliza la alerta. Así ningún error queda tragado sin registro: los nodos sin rama de error usan *Stop Workflow*.

Alcance del campo Content_ID: en la versión actual queda vacío en todos los registros, y es esperable. Los errores probados ocurren **antes** de crear la pieza (usuario no registrado, mensaje vacío, Fallo API en Busco Solicitante, Fallo AI), cuando todavía no existe un registro en Content al que vincularlos. Si falla un nodo **posterior** al alta (Cambio de Estado, mail HITL, updates o publicación), el Error Workflow registra el incidente como Fallo API con el link a la ejecución. Pero el Error Trigger solo recibe datos de la ejecución, no el `rec…` de la pieza, y por eso no puede completar Content_ID. Desde ese link se identifica la pieza afectada, que queda en estado Generando o En Revisión. Completar el vínculo automáticamente y marcar la pieza como Error queda como evolución hacia producción (§9). La consigna no lo exige: pide registrar el error, y eso se cumple.

5.3 Puntos Human-in-the-Loop

#	Punto	Qué controla
1	Confirmación explícita de la semilla por el solicitante	La IA reformula el tema y pregunta "¿Confirmás…?". Solo con un sí explícito pasa a generar. Evita gastar tokens y revisión en temas mal interpretados.
2	Aprobación del revisor por Gmail (punto crítico)	Antes de cualquier publicación, una persona lee el borrador completo y elige Aprobar o Rechazar.
3	Doble verificación en la base antes de publicar	El flujo vuelve a leer el registro y exige record id + Estado Aprobado + casilla Aprobado. Un error de ruteo no puede publicar una pieza no aprobada.

5.4 Tipos, comparaciones y anti-loop

- Todos los If usan **Type Validation: strict** y operadores acordes al tipo: boolean `is true` para `approved`, string `equals` para `estado = validado`, string `exists` / `not exists` para `id` y `bot_id`, y string `not empty` sobre valores con `trim()`.
- El filtro anti-loop se reforzó durante las pruebas. Al borrar mensajes, Slack envía eventos `message_deleted` sin `bot_id` que pasaban el filtro y generaban respuestas y logs en cascada. Se agregó la condición `!$json.subtype || $json.subtype === 'file_share'`: deja pasar mensajes nuevos e imágenes (necesarias para validar mensajes vacíos) y bloquea borrados, ediciones e ingresos al canal.
- Reintentos automáticos (**Retry on fail**) en AI Agent, Embeddings y Notion, los nodos más expuestos a fallos transitorios.

6. Entregable 5 · Dashboard de control

Nota: "Publicadas" y "Volumen de salida" cuentan piezas aprobadas por el revisor y entregadas por el canal de publicación simulada (mail), no posteos reales en Instagram.

El dashboard es una **Shared View pública** de la tabla KPIs. Se calcula solo con Recuento, Compilación y Fórmulas a partir de los vínculos de Content y Log de Errores, sin procesos extra.

Link público: https://airtable.com/appJBMsP4mvkNRtX8/shrcJgNW9fhIaMOKn/tblV3hHeQrxBDNDMO

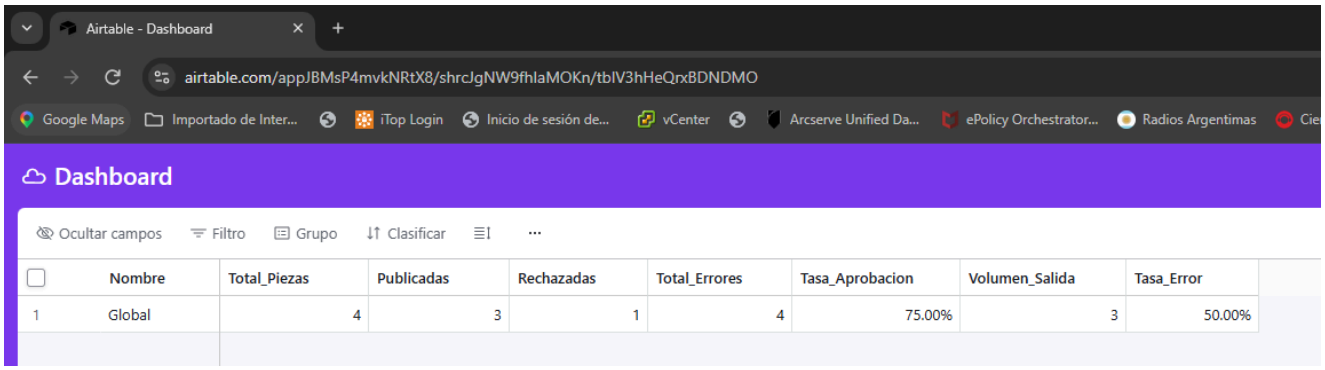


Figura 10. *Dashboard público de KPIs al cierre del stress test.*

KPI	Valor	Fórmula	Lectura
Tasa de aprobación	**75 %**	Publicadas / Total_Piezas = 3 / 4	De 4 piezas generadas, el revisor aprobó 3.
Volumen de salida	**3**	Publicadas	Posts que llegaron a publicarse.
Tasa de error	**50 %**	Total_Errores / (Total_Piezas + Total_Errores) = 4 / 8	Alta por diseño de la prueba: los 4 incidentes fueron forzados para validar cada rama de error.
Total de piezas · Rechazadas · Errores	4 · 1 · 4	Recuentos y compilaciones	Base de los indicadores anteriores.

> En operación normal se espera que la tasa de error baje mucho. Las validaciones previas existen justamente para que los errores de datos no lleguen a la IA, y cada incidente queda con link a la ejecución para diagnosticarlo.

7. Stress test y evidencias

Se ejecutaron 10 escenarios el 16/10/2026 sobre los flujos publicados, con las tablas limpias. Los fallos se provocaron desde Airtable o con un cambio temporal de parámetros, que luego se restauró.

#	Escenario	Cómo se provocó	Resultado	Evidencia
P1	Camino feliz · aprobado	Saludo → "Transferencia de un auto usado" → "Confirmo" → Aprobar	✅ Content #1 Publicado · ejecución #757	Fig. 11–15
P2	Camino feliz · 2° tema	"Verificación Policial" → Confirmo → Aprobar	✅ Content #2 Publicado	Fig. 16
P3	Rechazo HITL	"Formulario 08" → Confirmo → Rechazar	✅ Content #3 Rechazado · ejecución #777	Fig. 17–19
P4	Tema fuera de la base	"Cómo hacer un buen CV?"	✅ No inventa, redirige a temas válidos, sin registro	Fig. 20
P5	Usuario no registrado	Slack_User_ID modificado temporalmente	✅ Aviso en hilo · Log #1 Datos Faltantes · 0 tokens	Fig. 21
P6	Mensaje vacío	Imagen sin texto	✅ Aviso en hilo · Log #2 Datos Faltantes · 0 tokens	Fig. 22
P7	Fallo API	Campo Slack_User_ID renombrado temporalmente	✅ Ejecución #804 en error → Error WF #805 → alerta Slack · Log #3 Fallo API	Fig. 23–26
P8	Fallo AI	Max Tokens = 20 (respuesta truncada)	✅ Parser falla → error output → aviso + mail admin · Log #4 Fallo AI · ejecución #806	Fig. 27–29
P9	Anti-loop	Mensajes del propio bot / borrado de mensajes	✅ La ejecución se detiene en "Mensaje de Persona?" (#758)	Fig. 30
P10	Trazabilidad del dato	"Cédula verde" con el prompt dinámico nuevo	✅ Content #4 Publicado. Misma pieza de punta a punta: salida de la IA = Borrador_Generado; el filtro final exige Aprobado = TRUE()	Fig. 31–33

P1 · Camino feliz aprobado

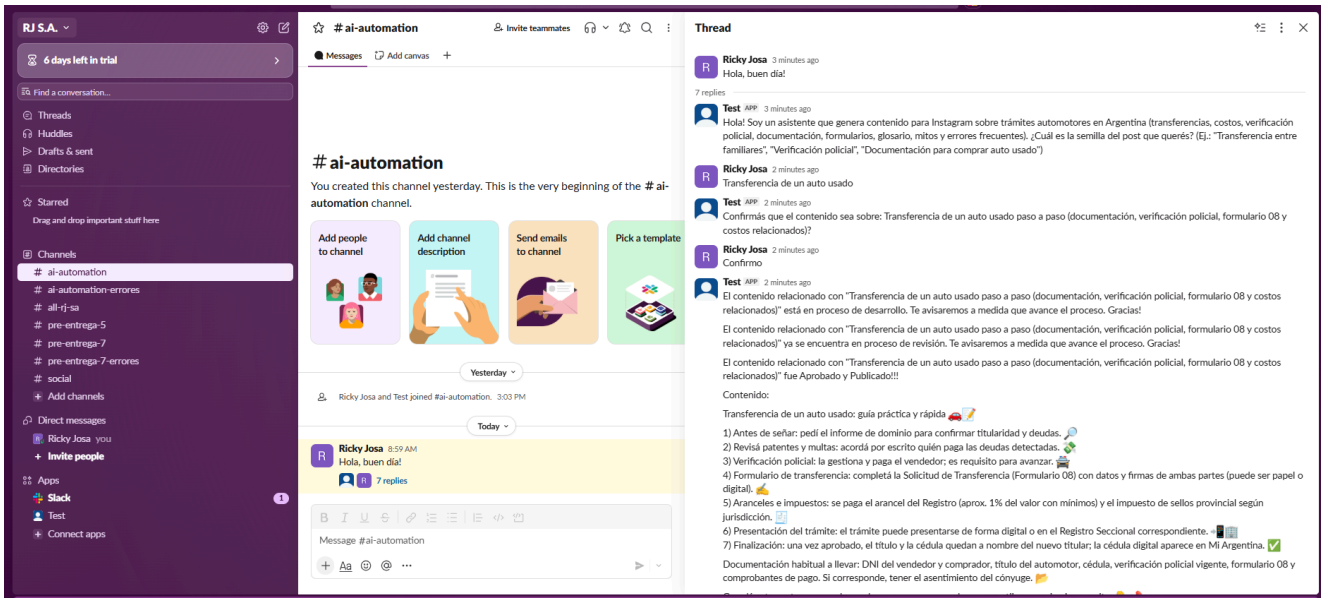


Figura 11. *P1 · Hilo completo en Slack: presentación, confirmación de la semilla, avisos de estado y post publicado.*

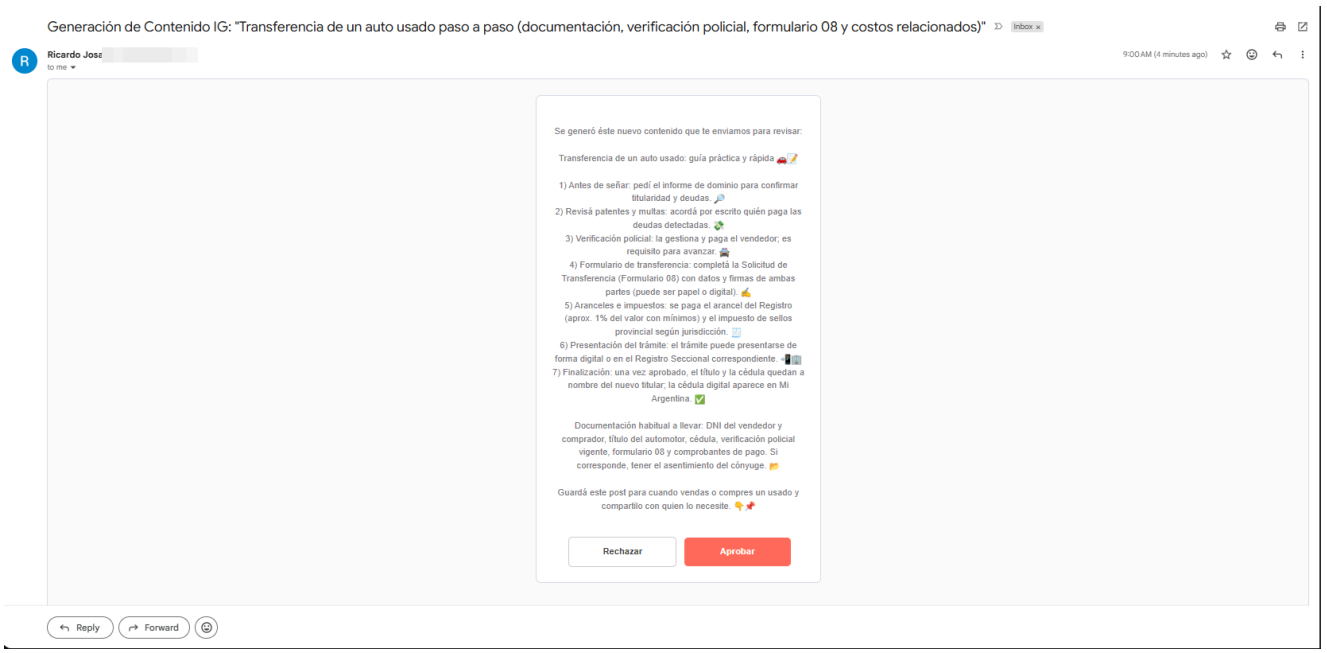


Figura 12. *P1 · Mail HITL con el borrador y los botones Aprobar / Rechazar.*

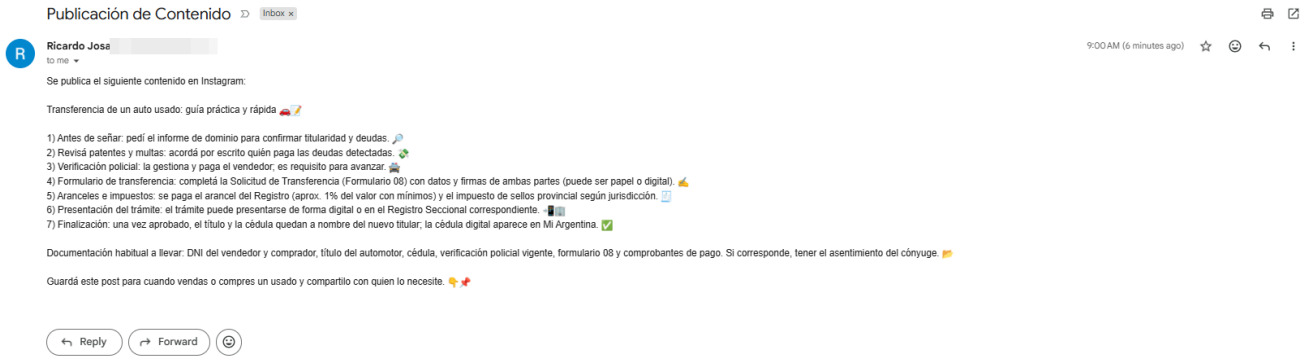


Figura 13. *P1 · Mail de publicación simulada en Instagram.*

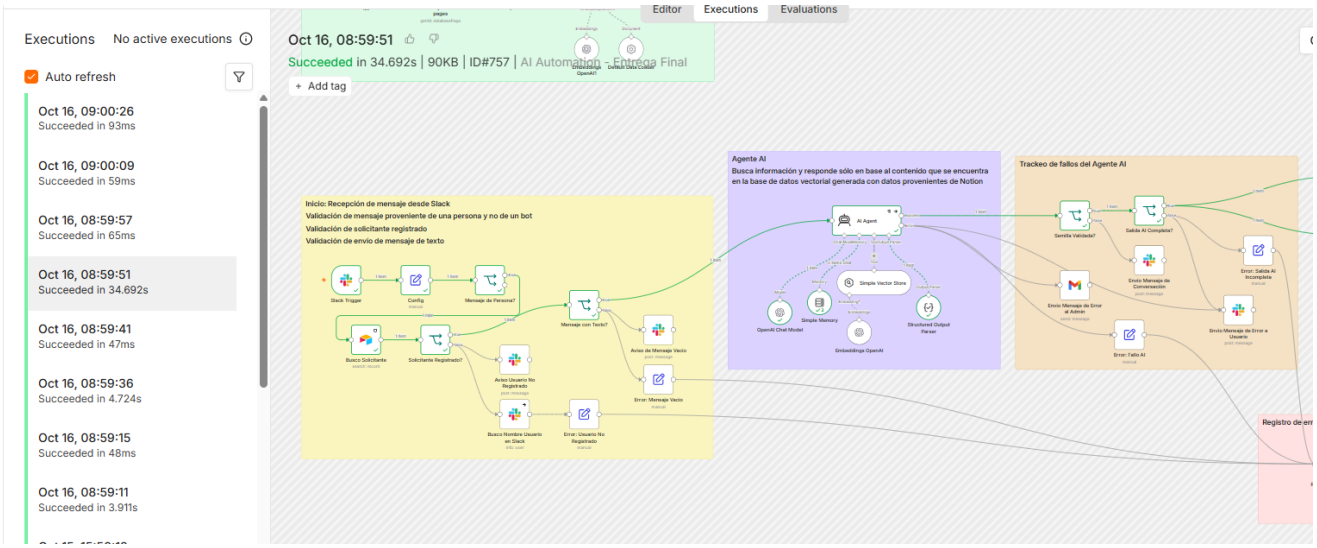


Figura 14. *P1 · Ejecución #757 (parte 1): validaciones, agente y control de salida.*

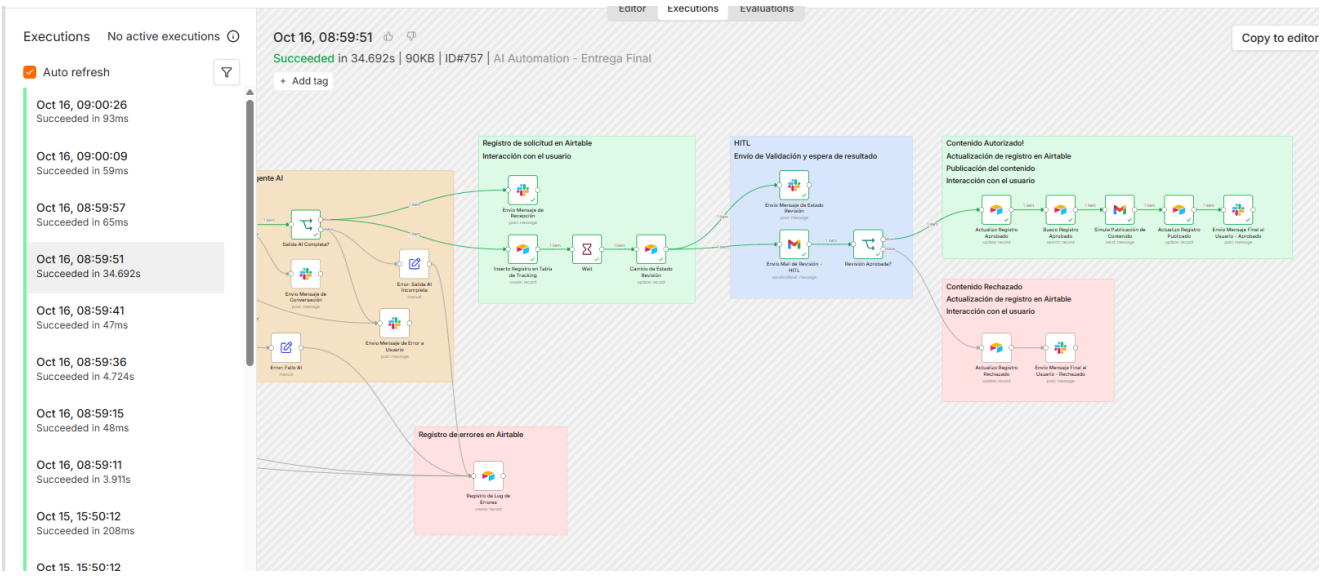


Figura 15. *P1 · Ejecución #757 (parte 2): registro, HITL y rama de contenido autorizado.*

P2 · Segundo camino feliz

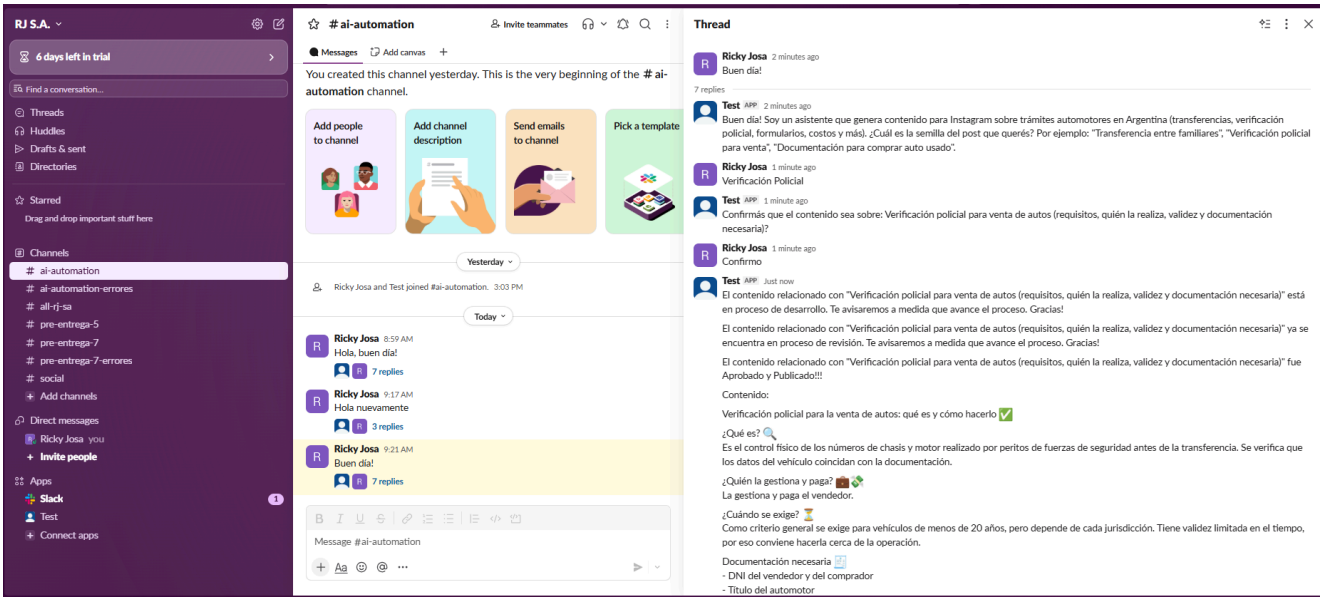


Figura 16. *P2 · Hilo de "Verificación policial" aprobado y publicado.*

P3 · Rechazo del revisor

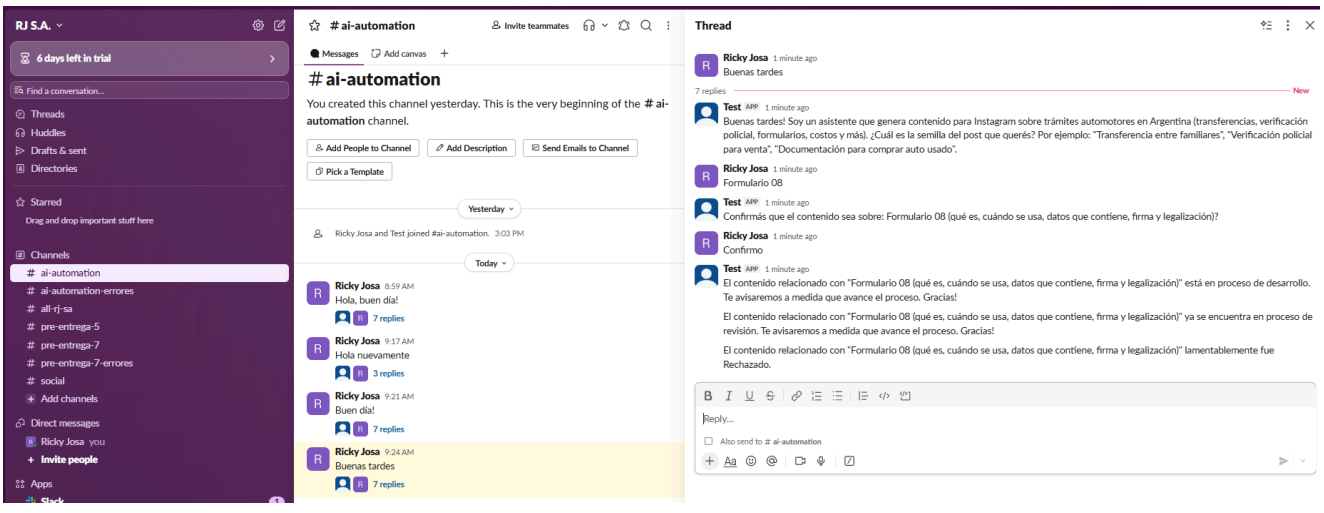


Figura 17. *P3 · Hilo de "Formulario 08" con aviso de rechazo.*

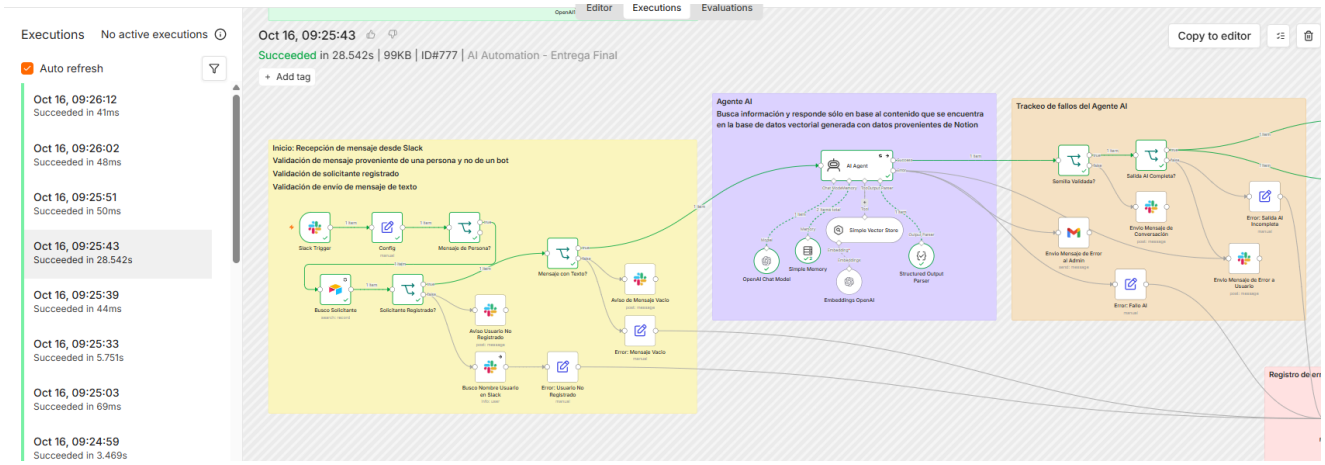


Figura 18. *P3 · Ejecución #777 (parte 1).*

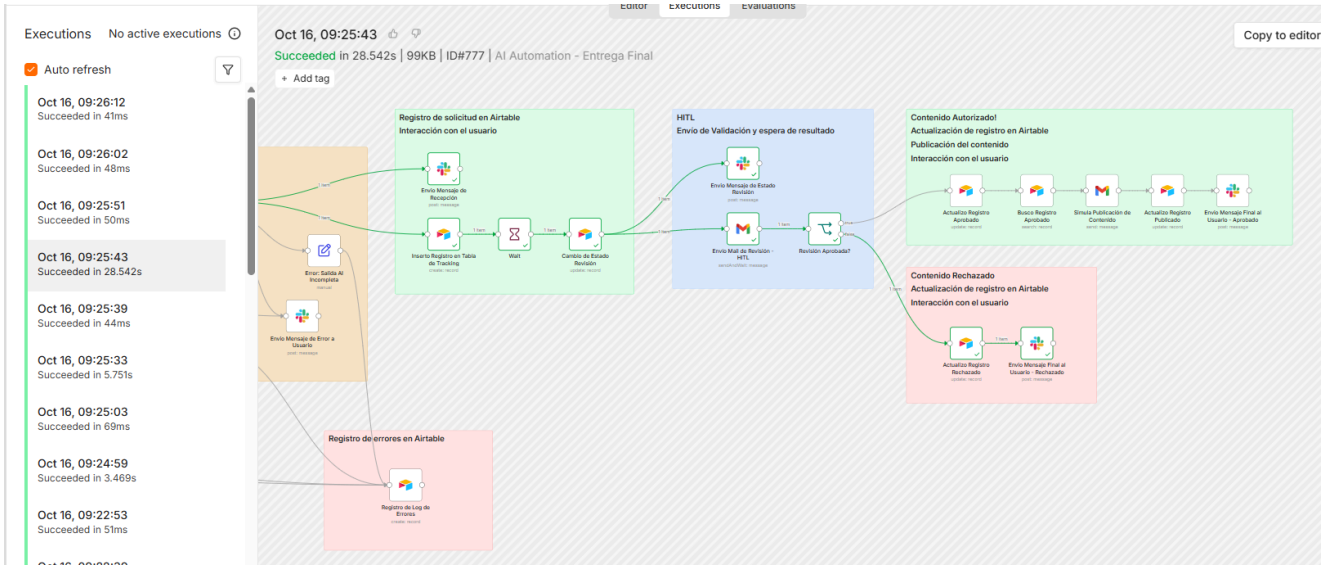


Figura 19. *P3 · Ejecución #777 (parte 2): la decisión HITL deriva a la rama de rechazo.*

P4 · Tema fuera de la base de conocimiento

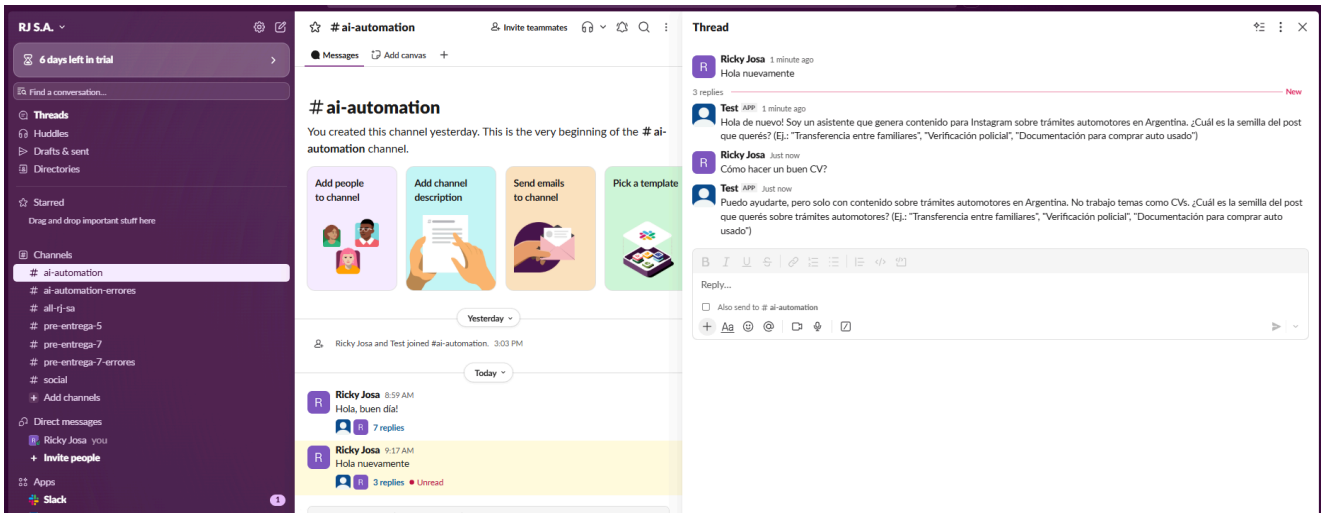


Figura 20. *P4 · El agente no inventa contenido fuera de su dominio y propone temas válidos.*

P5 y P6 · Validaciones previas a la IA

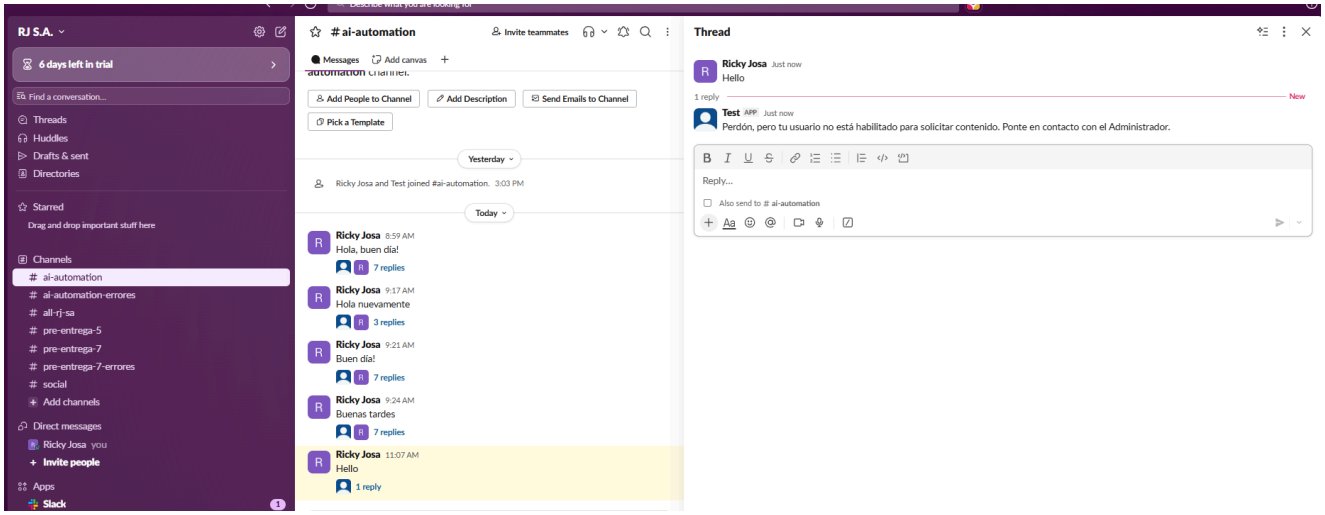


Figura 21. *P5 · Usuario no registrado: aviso en el hilo, sin consumo de tokens.*

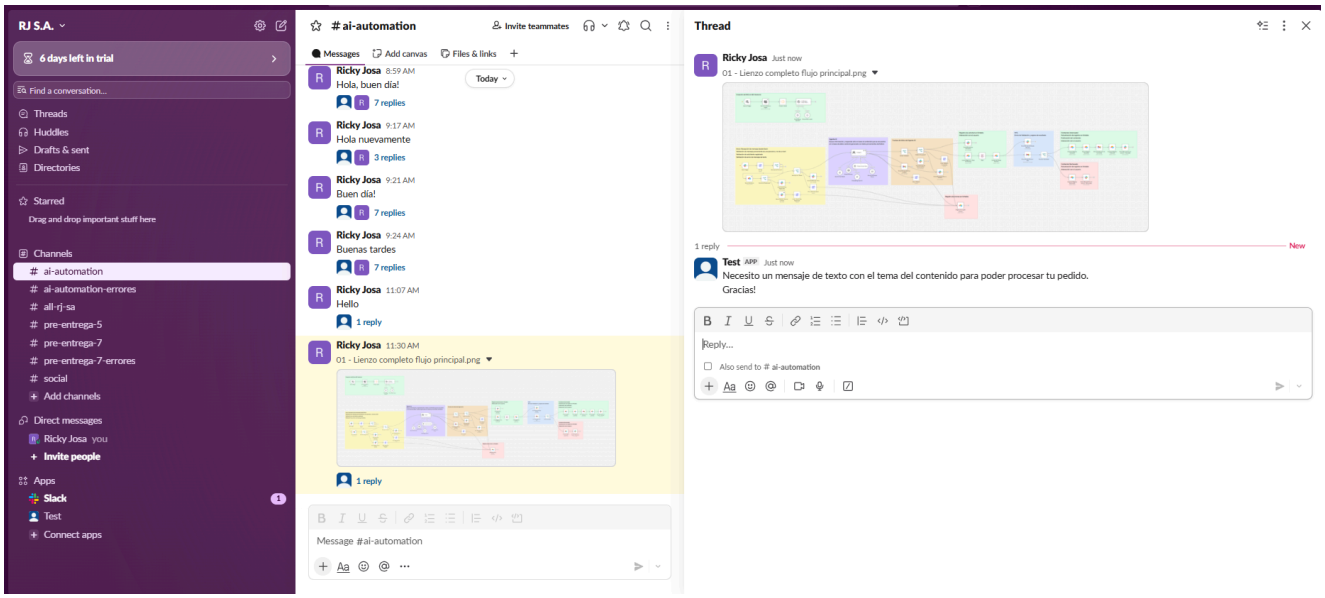


Figura 22. *P6 · Imagen sin texto: el flujo pide el tema en el hilo.*

P7 · Fallo de API capturado por el Error Workflow

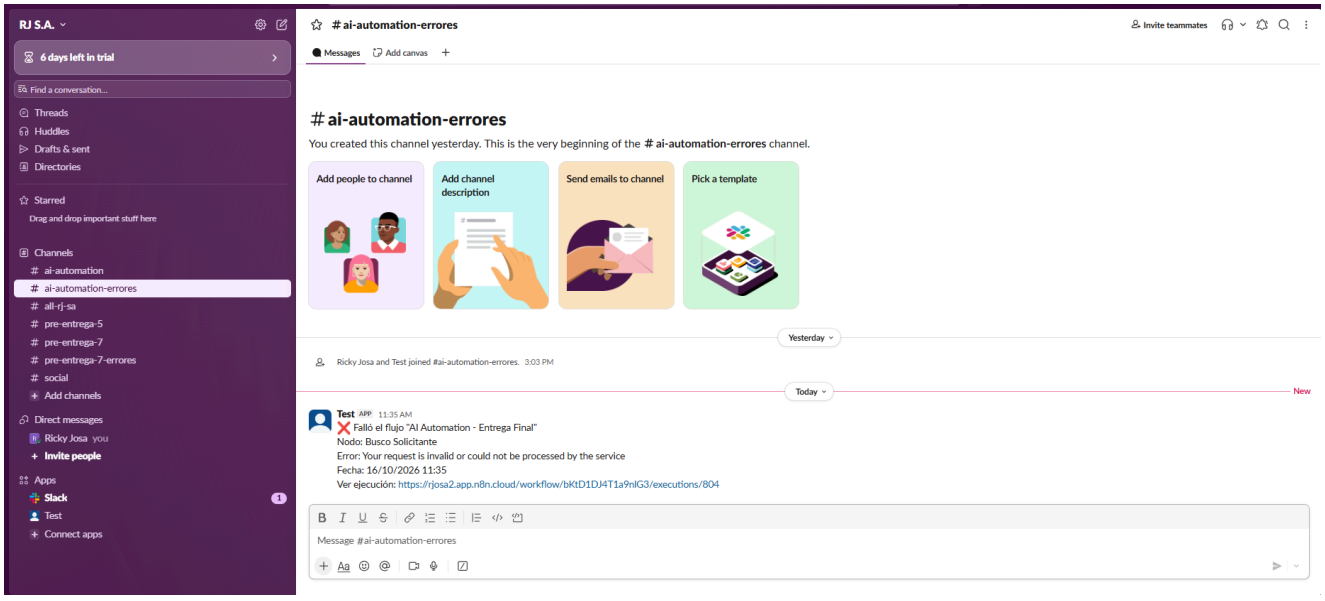


Figura 23. *P7 · Alerta en #ai-automation-errores con nodo, error, fecha y link a la ejecución.*

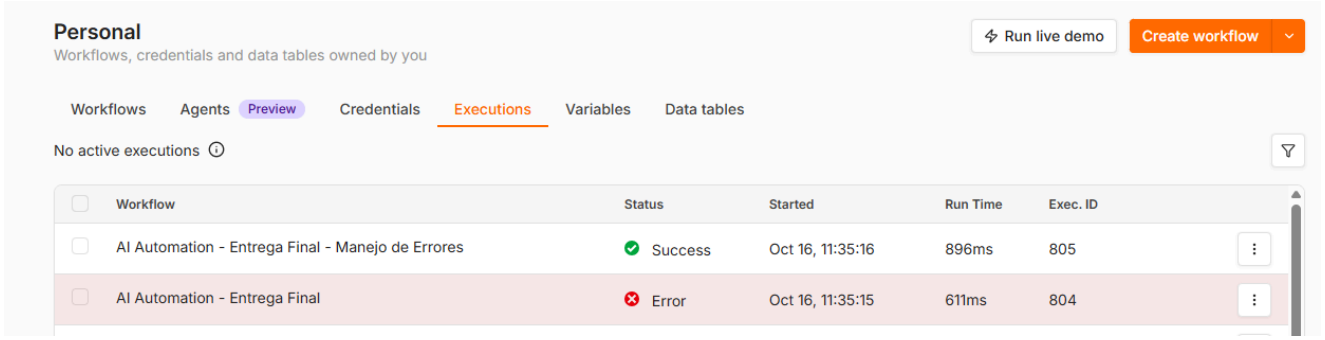


Figura 24. *P7 · Lista de ejecuciones: el flujo principal termina en error (#804) y dispara el Error Workflow (#805).*

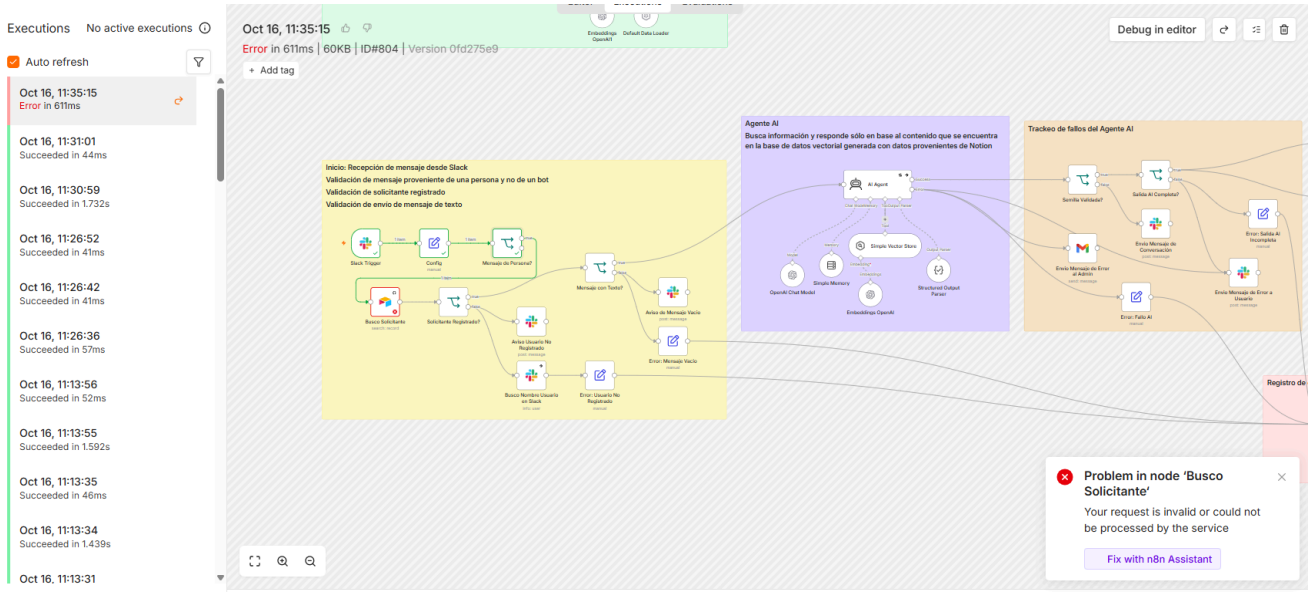


Figura 25. *P7 · Ejecución #804 detenida en "Busco Solicitante".*

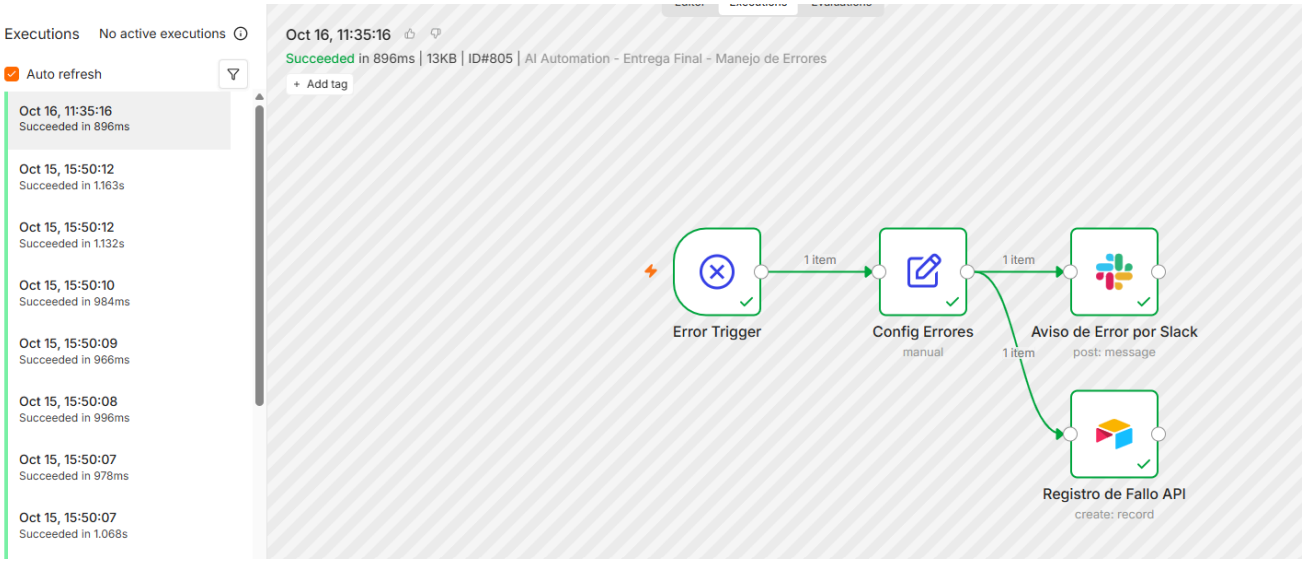


Figura 26. *P7 · Ejecución #805 del Error Workflow: alerta en Slack y registro en Airtable.*

P8 · Fallo de la IA controlado dentro del flujo

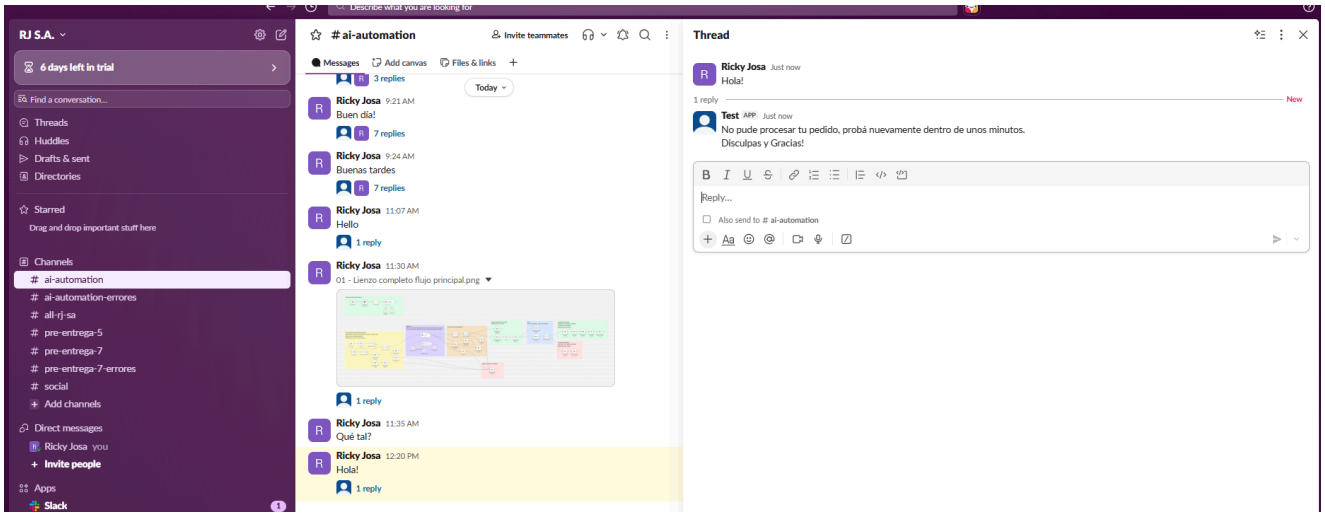


Figura 27. *P8 · Mensaje de error amable al usuario en el hilo.*



Figura 28. *P8 · Mail técnico al administrador con el detalle del error.*

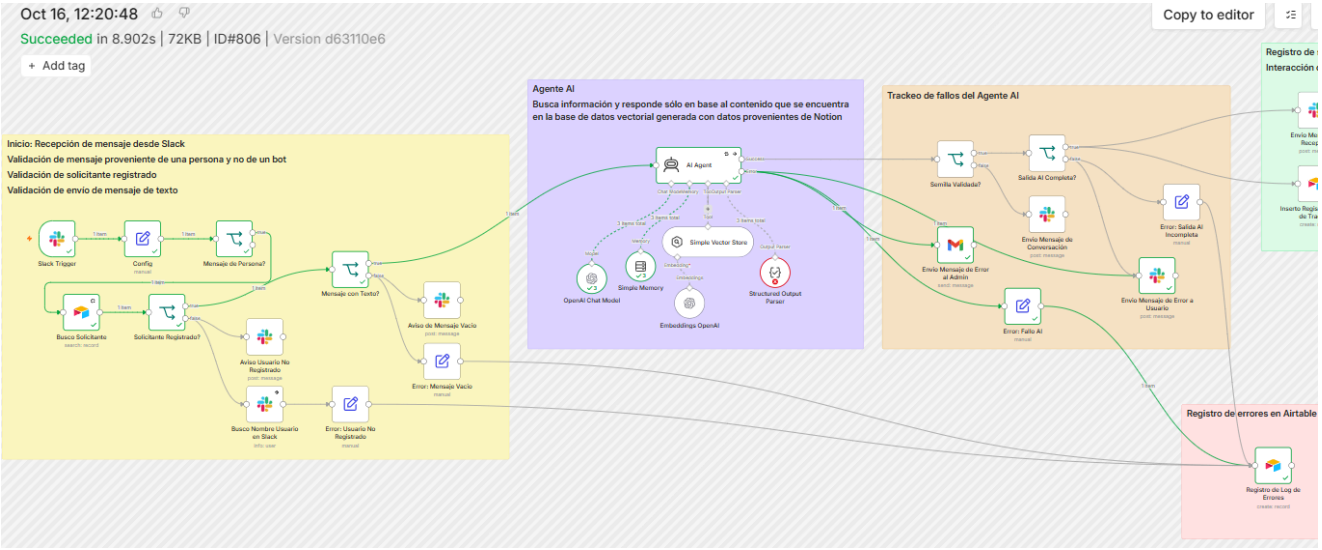


Figura 29. *P8 · Ejecución #806: el parser falla y el AI Agent sale por su salida de error hacia los avisos y el log.*

P9 · Filtro anti-loop

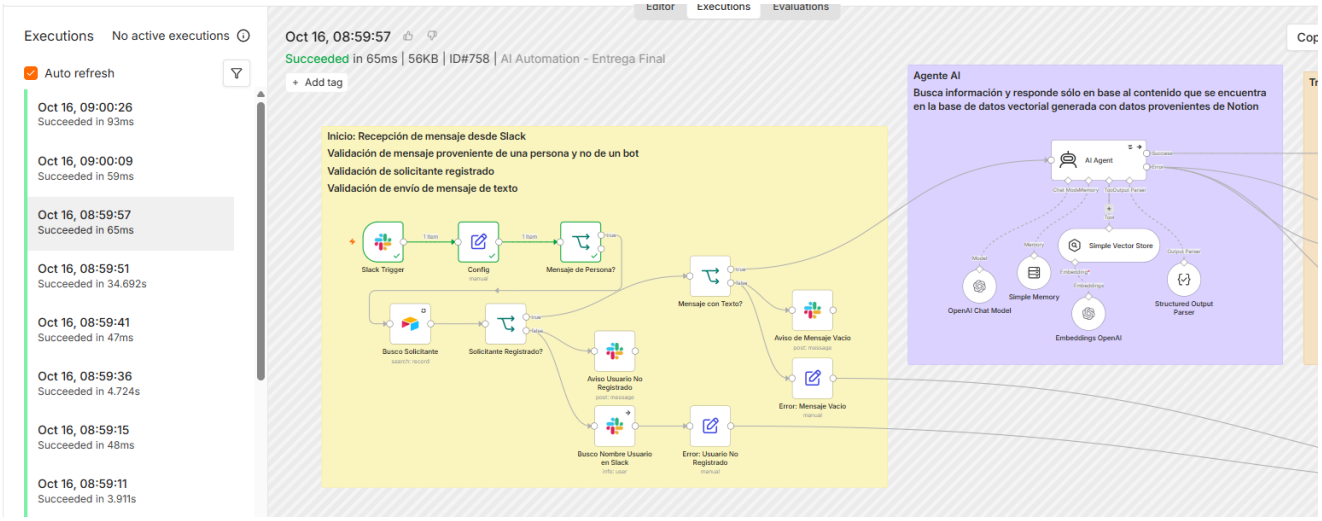


Figura 30. *P9 · Ejecución #758: un mensaje publicado por el propio bot se descarta en el primer If.*

P10 · Trazabilidad de una pieza de punta a punta (Content #4)

Esta prueba sigue un mismo dato a lo largo del flujo, ya con el prompt dinámico que incluye las variables del sistema. Muestra que el texto que genera la IA es exactamente el que se guarda, se revisa y se publica, y que la publicación solo ocurre si el registro cumple las tres condiciones del filtro final.

Paso	Nodo	Dato verificado
1 · Generación	AI Agent (salida del parser)	`estado: "validado"`, `semilla: "Cédula verde: qué es, mitos sobre vencimiento y duplicado"` y `contenido` con el post completo.
2 · Persistencia	Cambio de Estado Revisión	Registro `recLcCvV4xAOSoK5S` (ID 4): `Tema` = semilla, `Borrador_Generado` = contenido (mismo texto), `Estado = En Revisión`, vínculos a Solicitante y KPI, `Slack_Thread_TS` del hilo.
3 · Control previo a publicar	Busco Registro Aprobado	`AND(RECORD_ID() = 'recLcCvV4xAOSoK5S', {Estado} = 'Aprobado', {Aprobado} = TRUE())`: si alguna condición falla, la búsqueda no devuelve registros y el nodo de publicación no se ejecuta.

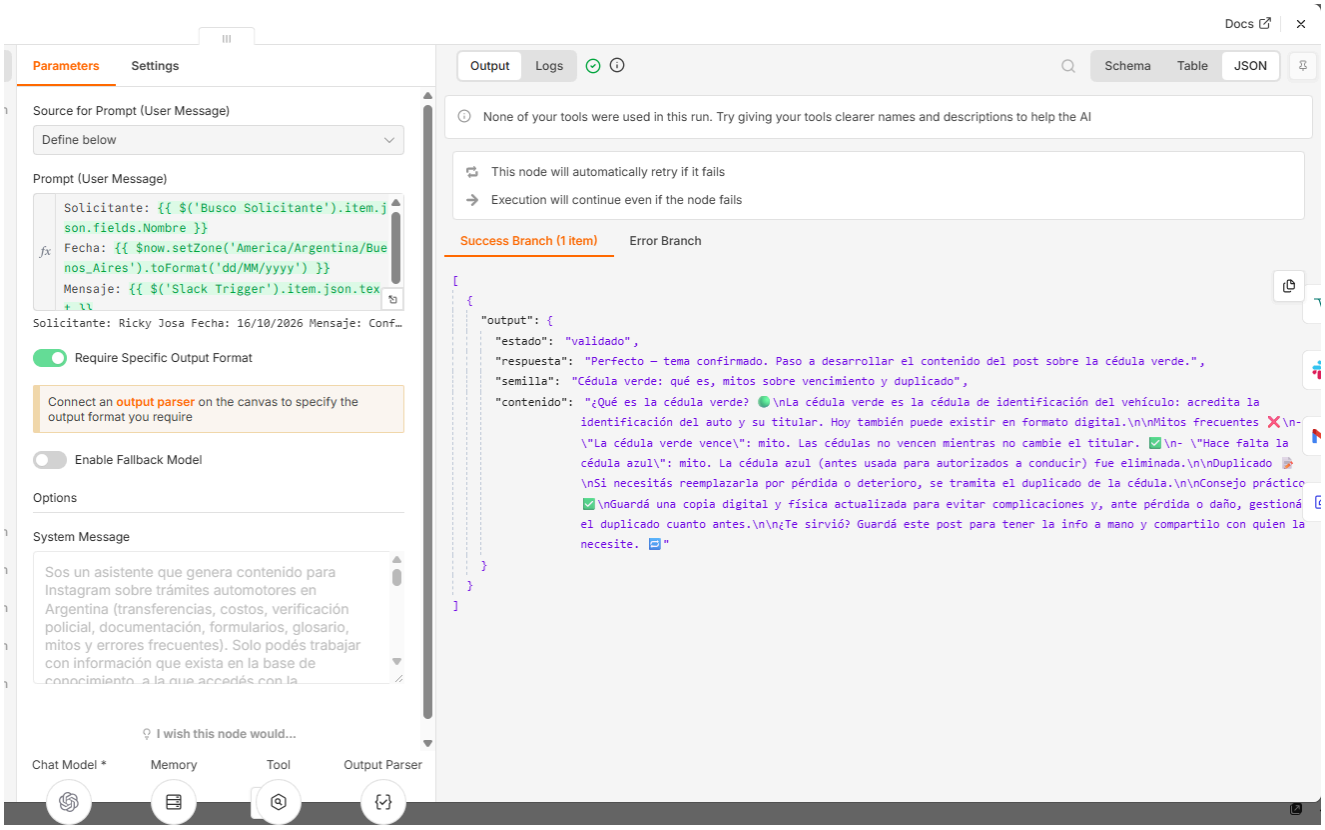


Figura 31. *P10 · Salida estructurada del AI Agent (JSON): estado, respuesta, semilla y contenido. El prompt muestra las variables Solicitante, Fecha y Mensaje ya resueltas.*

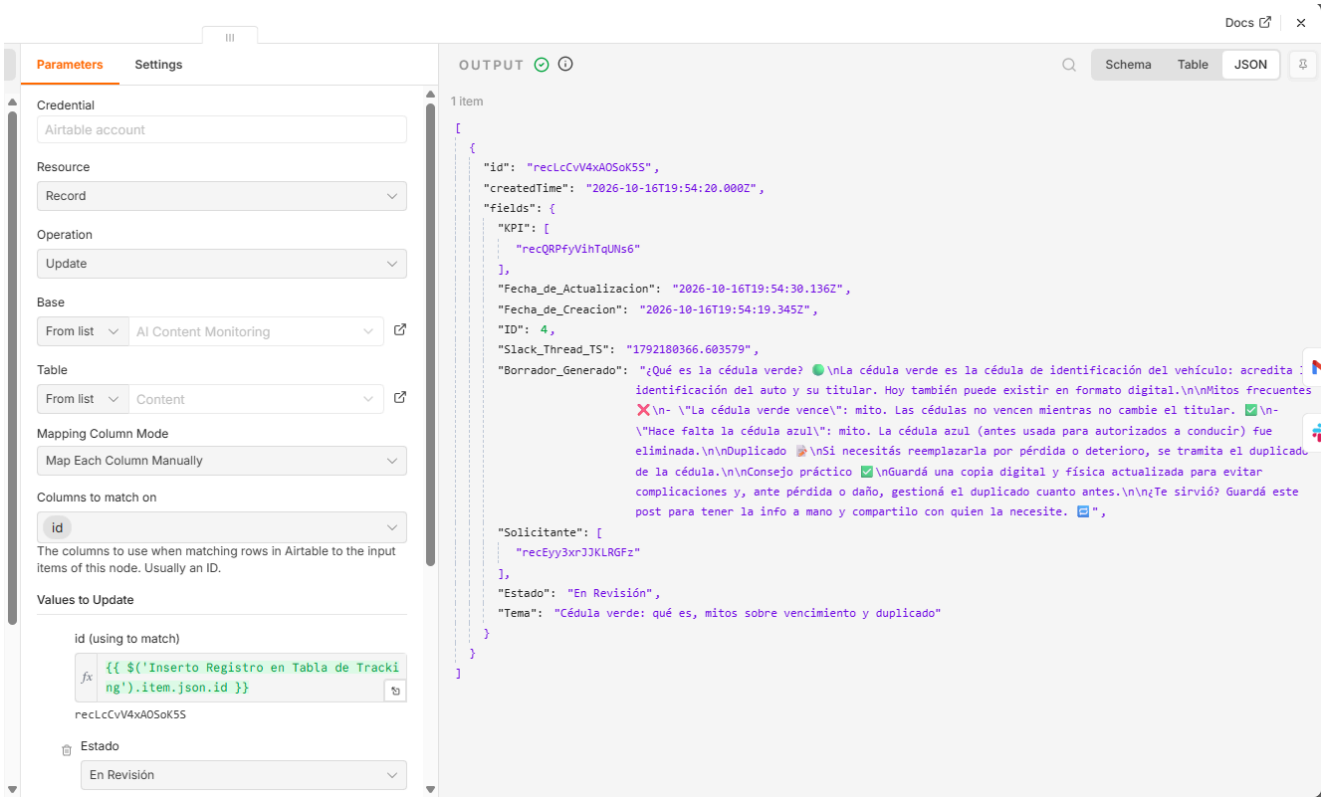


Figura 32. *P10 · Cambio de Estado Revisión: el campo Borrador_Generado guarda exactamente el contenido generado por la IA, con el mismo record id.*

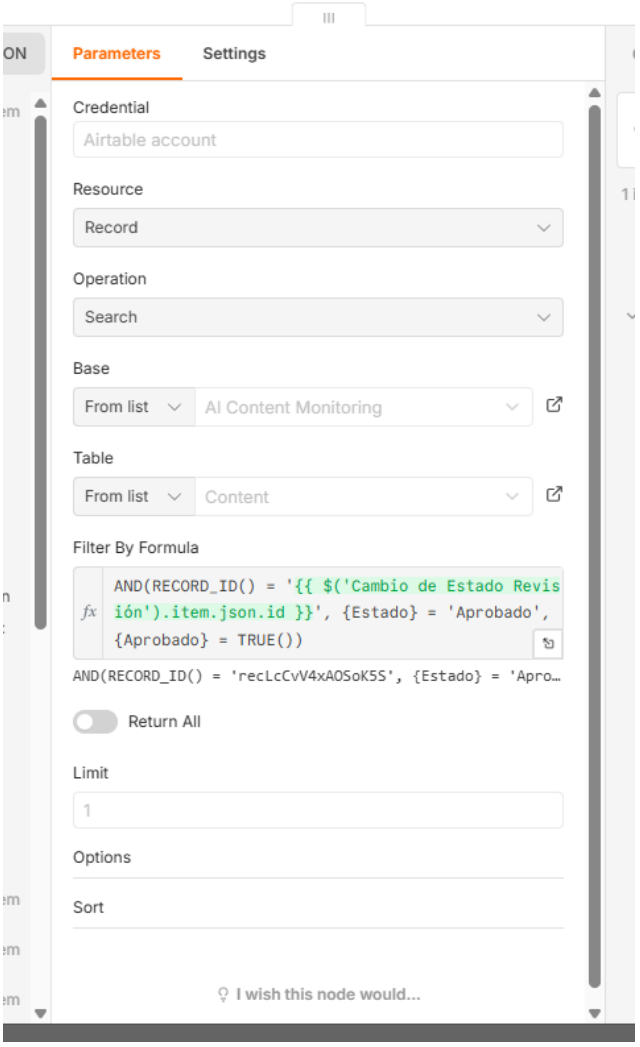


Figura 33. *P10 · Filtro final de Busco Registro Aprobado: record id + Estado = Aprobado + Aprobado = TRUE(). Debajo se ve la fórmula resuelta con el id real.*

Estado final de la base de datos

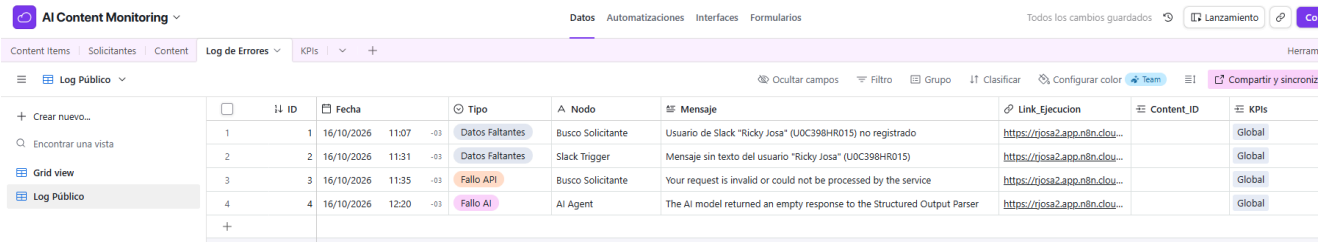


Figura 34. *Tabla Log de Errores: un registro por cada rama de error probada, con link a la ejecución.*

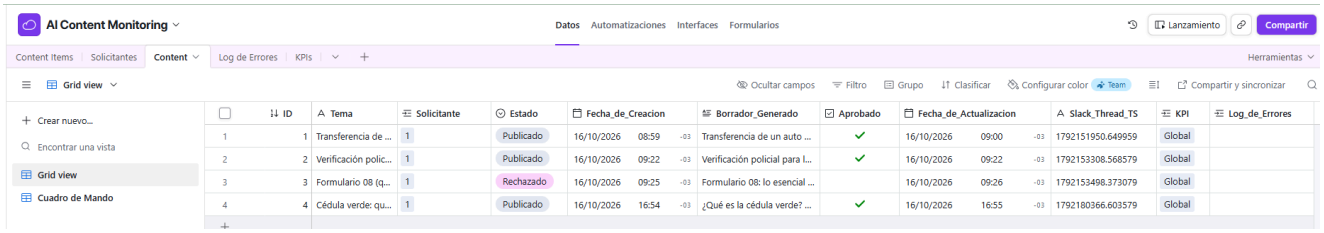


Figura 35. *Tabla Content: 3 piezas publicadas y 1 rechazada, con fechas, borrador, aprobación e ID de hilo.*

8. Matriz de cumplimiento de la consigna

Requisito de la consigna	Cumplimiento	Dónde
Orquestador n8n · Base de datos Airtable · IA OpenAI con agente y RAG · Canal de salida Slack y Gmail	✅ Las 4 categorías integradas	§2
El flujo se dispara, procesa con IA y entrega un output sin intervención manual	✅ Desde el mensaje en Slack hasta el aviso final; la única pausa es el HITL, que es obligatoria. La base RAG se re-indexa sola cada noche (Schedule Trigger)	§2.2
Campos de estado y relaciones entre tablas	✅ Estado con 6 valores + casilla Aprobado; 4 tablas vinculadas	§3.1 · §3.2
Trigger inteligente (evitar operaciones innecesarias)	✅ Trigger por canal + 3 filtros + validación de solicitante y texto antes de la IA	§2.2 · Fig. 7
Mapear la respuesta de la IA y limitar Max Tokens	✅ `output.*` por referencia explícita; Max Tokens 2000	§3.4 · §4
Error handling (Resume / Break) y registro si la API de IA falla	✅ Error output + Error Workflow; Log tipo Fallo AI probado	§5.2 · P8
Rutas de error ante datos faltantes o fallos de API	✅ Datos Faltantes (3 rutas) y Fallo API (Error Workflow)	§5.2 · P5–P7
HITL antes de la acción crítica	✅ Gmail Send & Wait + doble verificación en Airtable	§5.3
Thread ID de Slack mapeado	✅ `thread_ts || ts` guardado en Content	§3.3
Nodos nombrados claramente, variables dinámicas, sin hardcode	✅ Nombres descriptivos en español; Config y Config Errores	Fig. 3 · Fig. 6
Test de estrés ≥ 5 ejecuciones con camino infeliz	✅ 10 escenarios, 6 de camino infeliz	§7
Video demo de 3 min sin credenciales visibles	⏳ Link a completar	§2.3
Diagrama de arquitectura como imagen	✅ Dos vistas	§2.1
Manual operativo de datos + esquemas JSON explicados	✅	§3
Matriz de costos con modelo por tarea y ahorro estimado	✅ Incluye OpenAI, Claude y Batch API	§4
Seguridad: minimización + error handlers + HITL	✅	§5
Dashboard: Shared View pública con tasa de aprobación, volumen y tasa de error	✅ Verificado en ventana de incógnito	§6
JSON / Blueprint · link a la base en modo lectura · capturas de evidencia	✅ Links a Drive · Shared Views por tabla · §7	Portada · §7 · Anexo
Check de seguridad: anti-loop · tipos correctos · prompt dinámico	✅ ✅ ✅	§5.4 · Fig. 8

9. Evolución hacia producción

> **Fuera del alcance de esta entrega.** Los puntos de esta sección son mejoras pensadas para llevar el sistema a un uso productivo. Ninguno es un requisito de la consigna: todos los requisitos están cubiertos en la versión actual, como muestra la matriz de cumplimiento (§8). Algunos retoman sugerencias de la devolución de la pre-entrega 7.

- Publicación real en Instagram mediante la Graph API, en reemplazo del mail de simulación.
- Base RAG con fecha de actualización por entrada (propiedad en Notion, incluida como metadato en el nodo Preparar texto) para que el contenido recuperado sea trazable y se pueda detectar información vencida.
- Vector store persistente (por ejemplo, Supabase o Pinecone) para no depender de la memoria del servidor y re-indexar automáticamente cuando cambia Notion.
- Manejo explícito del timeout del HITL: recordatorio al revisor y estado "Vencido".

- Mover los valores de Config a Variables de entorno de n8n, para cambiar de ambiente sin editar el flujo.
- Ramas de error propias (*Continue using error output*) en los nodos posteriores al alta, para completar `Content_ID` en el Log de Errores y pasar la pieza a `Estado = Error`, en lugar de dejarla en Generando o En Revisión.

Anexo · Blueprints JSON

Los blueprints exportados desde n8n están en Google Drive con permiso de lectura para cualquier persona con el enlace. Para importarlos: n8n → Create workflow → ··· → Import from File.

Flujo	Link de descarga (Drive)
AI Automation - Entrega Final (principal) AI_Automation_-_Entrega_Final.json	https://drive.google.com/file/d/12SlWAtjaGD0JcJbtNbn9fmbJ5SbNKorn/view?usp=sharing
AI Automation - Entrega Final - Manejo de Errores AI_Automation_-_Entrega_Final_-_Manejo_de_Errores.json	https://drive.google.com/file/d/1kjR486NlzxmM_Rlqi5NV_Xrv6ebbM7Jy/view?usp=sharing

Después de importar hay que reasignar las credenciales (Slack, Gmail, Airtable, Notion, OpenAI), publicar el Error Workflow y seleccionarlo en Settings → Error Workflow del flujo principal.